

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Кубанский государственный университет»

Факультет компьютерных технологий и прикладной математики
Кафедра прикладной математики

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ)**

**РАЗРАБОТКА КРОСС-ПЛАТФОРМЕННОГО МОДУЛЯ
АВТОМОБИЛЬНОЙ СИМУЛЯЦИИ НА ОСНОВЕ
ДИНАМИЧЕСКОЙ БИБЛИОТЕКИ C++**

Работу выполнил _____ А.А. Мирошниченко
Направление подготовки 01.04.02 Прикладная математика и информатика

Научный руководитель

Нормоконтролёр

Краснодар
2025

СОДЕРЖАНИЕ

Введение.....	4
1 Сравнительный анализ математических моделей автомобильной покрышки	8
1.1 Требования к модели покрышки для симуляторов реального времени	8
1.2 Щёточная модель (Brush model).....	10
1.3 Модель Пасейки (Magic Formula).....	12
1.4 Модель Фиалы (Fiala).....	14
1.5 Модель Дугоффа (Dugoff).....	16
1.6 Существующие программные модули автомобильной физики.....	17
1.7 Выбор модели и выводы по главе.....	18
2 Математические модели узлов автомобиля.....	21
2.1 Модель покрышки и боковая сила трения.....	22
2.2 Независимая подвеска.....	24
2.3 Рулевое управление по принципу Аккермана.....	26
2.4 Дифференциал.....	26
2.5 Двигатель и коробка передач.....	27
3 Архитектура и программная реализация модуля.....	31
3.1 Общая архитектура библиотеки.....	31
3.2 Программный интерфейс на основе C-ABI.....	32
3.3 Численное интегрирование с фиксированным шагом.....	33
3.4 Структура программного кода.....	36
3.5 Кросс-платформенная сборка модуля.....	37
4 Интеграция в игровой движок и результаты тестирования.....	39
4.1 Интеграция в Unity.....	39
4.2 Оценка производительности.....	40
4.3 Оценка корректности и валидация модели.....	42
4.4 Ограничения и направления дальнейшей работы.....	44
Заключение.....	46

Список использованных источников.....	48
Приложение А. Программный интерфейс библиотеки на основе C-ABI.....	49
Приложение Б. Реализация математических моделей узлов автомобиля.....	53
Приложение В. Ядро симуляции и точка входа динамической библиотеки...	63
Приложение Г. Интеграция модуля в игровой движок Unity.....	70

ВВЕДЕНИЕ

Поведение автомобиля на дороге, привычное и интуитивно понятное любому водителю, в действительности рождается из сложного взаимодействия множества физических процессов, сосредоточенных в небольшом пятне контакта шины с дорожным полотном. Моделирование динамики автомобиля является одной из центральных задач в широком спектре прикладных областей — от разработки коммерческих автомобильных симуляторов и компьютерных игр до проектирования систем активной безопасности транспортных средств и создания обучающих тренажёров для подготовки водителей. Во всех этих областях достоверность виртуального поведения автомобиля определяет практическую ценность программного продукта, а центральным элементом, от которого зависит реалистичность модели в целом, служит математическое описание взаимодействия покрышки с дорожным полотном. Именно через пятно контакта шины с поверхностью реализуются все управляющие и возмущающие воздействия на транспортное средство, поэтому точность воспроизведения характеристик покрышки во многом предопределяет достоверность работы всего симулятора.

Современный рынок программных средств автомобильной симуляции можно условно разделить на три категории. К первой относятся профессиональные инженерные комплексы — такие как продукты семейства CarSim, CarMaker, а также решения на основе моделей шин MF-Tyre и MF-Swift. Эти системы обеспечивают высокую точность, верифицированную на экспериментальных данных, однако являются проприетарными, дорогостоящими и ориентированными на стендовое моделирование, а не на интеграцию в интерактивные приложения реального времени. Ко второй категории относятся игровые решения — физические подсистемы игровых движков и коммерческие дополнения, например Vehicle Physics Pro и NVIDIA PhysX Vehicle. Они доступны разработчикам, но жёстко привязаны к конкретному движку и зачастую используют упрощённые модели покрышки.

Третью категорию образуют проекты с открытым исходным кодом, среди которых выделяется библиотека Project Chrono; они открыты и научно обоснованы, но тяжеловесны и не предназначены для работы в реальном времени без значительной адаптации.

Анализ перечисленных решений выявляет незанятую нишу: отсутствует лёгкий, открытый и не привязанный к конкретному игровому движку программный модуль, который сочетал бы реалистичную полуэмпирическую модель покрышки с высокой вычислительной производительностью, достаточной для работы в реальном времени. Существующие модули автомобильной физики практически невозможно использовать вне той среды, для которой они были созданы, что вынуждает разработчиков повторно реализовывать физическую модель при переходе на новую платформу или новый движок. Это определяет актуальность настоящей работы.

Актуальность темы обусловлена растущим спросом на переносимые компоненты физического моделирования, которые могли бы использоваться без изменений в различных средах — игровых движках, научных вычислительных пакетах и собственных приложениях, — а также необходимостью достижения компромисса между реалистичностью поведения покрышки и вычислительными затратами при моделировании в реальном времени.

Объектом исследования является процесс математического моделирования динамики автомобиля в задачах симуляции реального времени. Предметом исследования выступают методы реализации физической модели автомобиля в виде обособленного кросс-платформенного программного модуля, а также способы интеграции такого модуля в игровые движки.

Целью работы является разработка кросс-платформенного модуля автомобильной симуляции, реализованного в виде самостоятельной динамически подключаемой библиотеки (DLL) на языке C++, обеспечивающего максимальную реалистичность поведения автомобиля при

высокой вычислительной производительности и независимости от конкретного игрового движка.

Для достижения поставленной цели в работе решаются следующие задачи:

- провести сравнительный анализ существующих математических моделей автомобильной покрышки и оценить их применимость в симуляторах реального времени;
- обосновать выбор математической модели покрышки и сопутствующих моделей узлов автомобиля (подвески, трансмиссии, рулевого управления, дифференциала);
- спроектировать архитектуру обособленного программного модуля и его программный интерфейс на основе чистого C-ABI, обеспечивающего совместимость с произвольными вызывающими средами;
- реализовать физическую модель автомобиля на языке C++ с применением численного интегрирования с фиксированным шагом;
- выполнить тестовую интеграцию модуля в игровой движок Unity и провести оценку производительности и корректности работы;
- сформулировать ограничения предложенного решения и направления дальнейшего развития.

Научная новизна работы заключается в следующем. Во-первых, предложена и реализована движко-независимая архитектура физической модели автомобиля, при которой полная полуэмпирическая модель покрышки и модели сопутствующих узлов инкапсулированы в обособленном бинарном модуле с чистым программным интерфейсом, что позволяет использовать единый код в различных игровых движках и вычислительных средах. Во-вторых, введён динамический коэффициент скольжения, моделирующий конечное время нарастания боковой силы трения покрышки и устраняющий нефизичное скатывание автомобиля с наклонных поверхностей, характерное для прямого применения статической формулы угла увода. В-третьих,

получена количественная оценка зависимости вычислительной нагрузки от частоты обновления физической модели, позволяющая обоснованно выбирать частоту интегрирования исходя из доступного бюджета кадра.

Практическая значимость работы состоит в том, что разработанный модуль может быть использован разработчиками игр, инженерами и исследователями в качестве готового переносимого компонента автомобильной физики, не требующего повторной реализации при смене платформы или среды исполнения. Модуль может быть собран под любую целевую платформу — персональные компьютеры, мобильные устройства, игровые консоли — при помощи соответствующего набора инструментов разработки.

Методологическую основу работы составляют положения теории динамики автомобиля и теории покрышки, методы численного интегрирования обыкновенных дифференциальных уравнений, а также принципы модульного проектирования программного обеспечения. При реализации использованы язык программирования C++ и игровой движок Unity в качестве тестовой платформы интеграции.

Структурно работа состоит из введения, четырёх глав, заключения и списка использованных источников. В первой главе рассматриваются существующие математические модели покрышки и приводится их сравнительный анализ. Во второй главе обосновываются и описываются математические модели, положенные в основу разрабатываемого модуля. В третьей главе излагаются архитектура программного модуля, его программный интерфейс и особенности численной реализации. В четвёртой главе приводятся результаты тестовой интеграции в игровой движок Unity и оценка производительности, а также формулируются ограничения и направления дальнейшей работы.

1 Сравнительный анализ математических моделей автомобильной покрышки

1.1 Требования к модели покрышки для симуляторов реального времени

Современная теория покрышки насчитывает несколько десятилетий развития и включает широкий спектр математических моделей — от простейших линейных приближений до сложных конечно-элементных расчётов. Для практического применения в симуляторах реального или квазиреального времени наибольший интерес представляют аналитические и полуэмпирические модели, способные обеспечить достаточную точность при приемлемых вычислительных затратах. Среди них наиболее широко известны модель Пасейки (Magic Formula), щёточная модель (Brush model), модель Фиалы (Fiala) и модель Дугоффа (Dugoff).

Прежде чем перейти к описанию конкретных моделей, необходимо сформулировать критерии, которым должна удовлетворять математическая модель покрышки, предназначенная для использования в автомобильном симуляторе. Первым и наиболее очевидным требованием является вычислительная эффективность. Симуляторы, работающие в режиме реального времени, как правило, требуют обновления физической модели с частотой от 100 до 1000 Гц, что накладывает жёсткие ограничения на сложность вычислений, выполняемых на каждом шаге интегрирования. Чем выше частота обновления, тем меньше времени отводится на расчёт одного шага, и тем критичнее становится вычислительная сложность модели.

Вторым важным требованием является точность воспроизведения нелинейных эффектов. Поведение покрышки существенно нелинейно: при малых углах увода боковая сила растёт почти линейно, однако по мере увеличения угла прирост замедляется, и в конечном счёте сила достигает насыщения. Аналогичный характер имеет зависимость продольной силы от скольжения. В режиме экстремального вождения, к которому относится,

например, участие в виртуальных гонках, нелинейная область является основной рабочей зоной, поэтому её адекватное воспроизведение принципиально важно.

Третьим требованием служит способность модели корректно описывать комбинированное нагружение, то есть одновременное действие продольных и боковых сил. Именно такой режим наиболее распространён при реальном движении автомобиля — торможение в повороте, разгон при выходе из поворота и так далее. Раздельное рассмотрение продольной и боковой динамики приводит к завышению суммарной силы, реализуемой покрышкой, и, как следствие, к нефизичному поведению автомобиля на пределе сцепления.

Наконец, важны также возможность идентификации параметров модели по стандартным измерительным данным и удобство её интеграции в многотельные динамические системы. Эти требования вступают в противоречие друг с другом: повышение точности обычно требует усложнения модели и расширения экспериментальной базы, тогда как повышение производительности и простоты идентификации достигается за счёт упрощений. Выбор конкретной модели представляет собой поиск компромисса между этими противоречивыми требованиями.

Следует отдельно отметить специфику применения моделей покрышки именно в интерактивных приложениях, отличающую их от инженерных расчётных задач. В инженерном моделировании, выполняемом, как правило, в отложенном режиме, допустимо затрачивать значительное время на расчёт одного шага, добиваясь максимальной точности. В интерактивном симуляторе ситуация принципиально иная: расчёт физики должен завершаться в пределах жёстко ограниченного бюджета кадра, разделяемого с подсистемами отрисовки, обработки ввода и игровой логики. Кроме того, в интерактивном приложении недопустима потеря устойчивости численной схемы даже в редких граничных режимах — таких как движение на очень малой скорости, мгновенное изменение поверхности или столкновение, — поскольку любая расходимость немедленно проявляется в виде заметного пользователю

артефакта. Эти соображения существенно сужают круг практически применимых моделей и выдвигают на первый план не только точность, но и численную устойчивость и предсказуемость поведения модели во всём диапазоне рабочих режимов.

Помимо собственно силовых характеристик, полноценная модель покрышки для симулятора должна корректно воспроизводить переходные процессы. Реакция шины на изменение угла увода не является мгновенной: боковая сила нарастает по мере прохождения покрышкой некоторого пути, что характеризуется длиной релаксации. Игнорирование переходных процессов приводит к чрезмерно резкой, «дёрганой» реакции автомобиля на управляющие воздействия и к ряду нефизичных эффектов при движении на малых скоростях. Поэтому при выборе и реализации модели необходимо предусмотреть механизм учёта конечного времени нарастания силы трения, даже если базовая форма выбранной модели описывает лишь установившийся режим.

1.2 Щёточная модель (Brush model)

Щёточная модель является одной из наиболее физически обоснованных аналитических моделей покрышки. Она представляет протектор шины в виде набора упругих элементов — «щетинок», — равномерно распределённых по длине пятна контакта. Каждая щетинка одним концом прикреплена к жёсткому ободу покрышки, а другим касается дорожного покрытия. При движении автомобиля щетинки последовательно входят в пятно контакта на переднем крае, деформируются под действием трения и покидают его на заднем крае. Сила, действующая на шину, определяется как интеграл сдвиговых напряжений по площади пятна контакта.

Щёточная модель строится на нескольких упрощающих предположениях. Распределение нормального давления по длине пятна контакта принимается параболическим или равномерным. Коэффициент трения считается постоянным и изотропным. Упругие характеристики

щетинок описываются линейной зависимостью. При этих допущениях модель допускает аналитическое решение и позволяет получить явные выражения для боковой и продольной сил, а также момента самовыравнивания в зависимости от угла увода и продольного скольжения.

Достоинством щёточной модели является её физическая прозрачность: каждый параметр имеет чёткий физический смысл — жёсткость протектора, длина пятна контакта, нормальная нагрузка, коэффициент трения. Это позволяет относительно просто идентифицировать параметры по результатам испытаний и качественно понять, каким образом те или иные конструктивные изменения влияют на характеристики покрышки. Недостатком модели является то, что она недостаточно точно воспроизводит реальное поведение шины при больших углах увода и высоких нагрузках, поскольку действительное распределение давления и трения в пятне контакта значительно сложнее принятых допущений.

В практике автомобильного моделирования щёточная модель применяется там, где требуется хорошее понимание физики процессов, а не предельная точность воспроизведения характеристик конкретной шины. Она особенно полезна на этапе концептуального проектирования и при создании учебных симуляторов, в которых важна общая достоверность поведения автомобиля, а не соответствие характеристикам определённой марки покрышки.

С вычислительной точки зрения щёточная модель в её аналитической форме умеренно затратна: получение явных выражений для сил требует вычисления нескольких тригонометрических и степенных функций, однако не предполагает численного интегрирования по площади пятна контакта. Это делает её пригодной для применения в реальном времени, хотя и менее экономичной, чем простейшие аппроксимации. Отдельным достоинством щёточной модели является то, что она естественным образом описывает комбинированное нагружение: продольная и боковая деформации щетинок рассматриваются совместно, а условие проскальзывания учитывает их

суммарное действие. Тем самым модель внутренне согласована и не требует дополнительных эвристик для ограничения результирующей силы.

1.3 Модель Пасейки (Magic Formula)

Модель Пасейки, широко известная под названием Magic Formula («волшебная формула»), была разработана голландским учёным Хансом Бертом Пасейкой в конце 1980-х годов совместно с компанией Volvo Car. Первоначально она создавалась как чисто эмпирический инструмент для аппроксимации экспериментально измеренных характеристик шины, однако со временем стала де-факто стандартом в области моделирования покрышек и используется практически всеми ведущими мировыми производителями автомобилей и гоночными командами.

В основе модели лежит математическая зависимость, связывающая выходные силы и моменты с кинематическими параметрами шины — углом увода, продольным скольжением и углом развала. В наиболее распространённой базовой форме зависимость нормированной силы от обобщённого параметра скольжения записывается в виде:

$$y = D \cdot \sin \{ C \cdot \arctan [B \cdot x - E \cdot (B \cdot x - \arctan(B \cdot x))] \} \quad (1.1)$$

где x — обобщённый параметр скольжения (угол увода либо продольное скольжение), а коэффициенты B , C , D и E задают, соответственно, жёсткость, форму, пиковое значение и кривизну характеристики. Коэффициент D определяет максимальное значение силы и пропорционален нормальной нагрузке; коэффициент C отвечает за общую форму кривой; произведение $B \cdot C \cdot D$ задаёт наклон характеристики в начале координат; коэффициент E корректирует положение и форму области насыщения. Формула имеет S-образный характер и описывает поведение покрышки от нулевого значения нагрузки до насыщения при предельных углах и скольжениях.

Параметры модели не имеют прямого физического смысла — они определяются путём аппроксимации экспериментальных данных методом наименьших квадратов. Именно поэтому модель называют «волшебной»: она

точно воспроизводит измеренные характеристики, но не объясняет их физическую природу. Версия модели, обозначаемая как MF 5.2 или более поздняя MF-Swift, включает описание как установившихся, так и переходных режимов нагружения покрышки. Переходные процессы особенно важны для моделирования управляемости, поскольку реакция шины на изменение угла увода не является мгновенной — она характеризуется так называемой длиной релаксации. Кроме того, расширенные версии модели учитывают зависимость коэффициента трения от температуры и скорости, что делает их применимыми в задачах моделирования гоночных шин, существенно изменяющих свои свойства в ходе заезда.

Главным достоинством модели Пасейки является исключительная точность воспроизведения характеристик конкретных шин при наличии полного набора экспериментальных данных. Именно это обстоятельство сделало её стандартным инструментом в профессиональных симуляторах гоночных автомобилей, системах помощи водителю и программном обеспечении для разработки шасси. Основным недостатком является зависимость от обширной экспериментальной базы: точная идентификация параметров полной формулы, в которой может насчитываться более сорока коэффициентов, требует дорогостоящих испытаний на специальных стендах. Помимо этого, модель не обладает физической интерпретируемостью и плохо экстраполирует за пределы диапазона измерений.

Важно, однако, что модель Пасейки допускает применение в усечённой, базовой форме, в которой число коэффициентов сокращается до четырёх на каждое направление. В этом случае модель утрачивает способность точно воспроизводить характеристики конкретной шины, но сохраняет правильную качественную форму зависимости и остаётся вычислительно чрезвычайно экономичной. Коэффициенты базовой формы могут быть подобраны эмпирически — исходя из желаемого пикового значения силы, угла его достижения и общей формы кривой — без проведения стендовых испытаний. Такой подход особенно уместен в игровых симуляторах, где целью является

не точное соответствие конкретной шине, а правдоподобное и управляемое поведение автомобиля. Именно эта особенность делает модель Пасейки практически безальтернативной для рассматриваемой задачи: она сочетает минимальную вычислительную стоимость одного вызова с физически корректной нелинейной формой характеристики.

Отдельного внимания заслуживает связь коэффициентов формулы с наблюдаемыми характеристиками покрышки. Коэффициент D , задающий пиковое значение силы, физически соответствует произведению коэффициента сцепления на нормальную нагрузку, и потому в реализации он естественным образом масштабируется силой подвески. Произведение $B \cdot C \cdot D$ определяет начальный наклон характеристики — так называемую жёсткость увода или жёсткость продольного скольжения, — которая отвечает за отзывчивость автомобиля при малых управляющих воздействиях. Зная требуемые пиковое значение и угол его достижения, коэффициент жёсткости B можно вычислить аналитически, что и реализовано в модуле при инициализации модели покрышки.

1.4 Модель Фиалы (Fiala)

Модель Фиалы, предложенная в 1950-х годах и носящая имя своего создателя Эрнста Фиалы, представляет собой аналитическую модель, занимающую промежуточное положение между простейшими линейными приближениями и детализированными физическими моделями. Подобно щёточной, она основана на рассмотрении деформаций протектора в пятне контакта, однако использует иное распределение давления и иную механику скольжения, что приводит к несколько другим аналитическим выражениям для боковой силы и момента самовыравнивания.

Особенностью модели Фиалы является то, что она явно выделяет область прилипания и область скольжения внутри пятна контакта. В области прилипания деформация протектора нарастает линейно от переднего края контакта к заднему, а в области скольжения деформация ограничивается

условием Кулона. Такой подход позволяет аналитически описать переход от линейного поведения при малых углах увода к насыщению при больших. Модель хорошо подходит для описания боковых сил, однако её возможности по воспроизведению совместного действия продольных и боковых сил ограничены.

Модель Фиалы широко применяется в многотельных динамических системах, в том числе в среде MSC Adams, где она реализована как стандартная опция. Её параметры — угловая жёсткость, нормальная нагрузка, коэффициент трения и геометрия пятна контакта — могут быть получены из стандартных справочных данных без проведения специализированных испытаний. Это делает модель привлекательной для задач, где нет необходимости в предельной точности, но важна физическая обоснованность результатов. Ограничением служит меньшая точность по сравнению с моделью Пасейки при описании поведения шины вблизи предела сцепления.

С точки зрения вычислительной сложности модель Фиалы сопоставима с щёточной и заметно проще полной модели Пасейки. Однако её существенным недостатком применительно к задачам интерактивной симуляции является ограниченность описания комбинированного нагружения: модель ориентирована прежде всего на расчёт боковой силы и момента самовыравнивания, тогда как совместное действие продольной и боковой сил описывается ею менее последовательно. В симуляторе, где автомобиль постоянно находится в режимах одновременного разгона или торможения и поворота, это ограничение оказывается практически значимым и требует введения дополнительных корректирующих механизмов, что снижает привлекательность модели по сравнению с подходом, в котором комбинированное нагружение учитывается единообразно через эллипс трения.

1.5 Модель Дугоффа (Dugoff)

Модель Дугоффа была предложена в 1969 году и является одной из первых попыток создать единое аналитическое описание продольных и боковых сил шины в комбинированном режиме нагружения. Модель строится на нескольких упрощающих предположениях об эллиптической форме пятна контакта и постоянном коэффициенте трения. Она включает функцию насыщения, которая при предельных нагрузках ограничивает значение результирующей силы величиной, определяемой коэффициентом трения и нормальной нагрузкой.

Несмотря на относительную простоту, модель Дугоффа корректно воспроизводит ряд ключевых особенностей поведения шины: рост продольной и боковой сил при малых значениях скольжения и угла увода, насыщение при больших значениях, а также взаимное влияние продольного и бокового скольжений. Последнее обстоятельство делает её более предпочтительной по сравнению с моделями, рассматривающими продольную и боковую динамику независимо, особенно в задачах моделирования торможения в повороте и оценки эффективности систем ABS и ESP.

Главным недостатком модели Дугоффа является её неточность при больших углах развала и в условиях переменного коэффициента трения. Кроме того, она не учитывает пневматический след и момент самовыравнивания, что ограничивает её применимость для моделирования рулевого управления. Тем не менее благодаря вычислительной простоте и наличию явных аналитических выражений модель по-прежнему используется в задачах, требующих высокой скорости вычислений, например в бортовых системах управления автомобилем и в симуляторах с жёсткими ограничениями по производительности.

1.6 Существующие программные модули автомобильной физики

Помимо математических моделей покрышки, рассмотренных выше, целесообразно проанализировать существующие готовые программные решения, реализующие автомобильную физику, поскольку именно с ними сопоставляется разрабатываемый модуль. Условно эти решения можно разделить на три группы: профессиональные инженерные комплексы, игровые решения и проекты с открытым исходным кодом.

К профессиональным инженерным комплексам относятся, в частности, продукты на основе моделей MF-Tyre и MF-Swift, разработанных в рамках линейки решений по динамике шин, а также комплексы CarSim и CarMaker, предназначенные для полного моделирования динамики автомобиля. Эти решения обеспечивают высочайшую точность, верифицированную на экспериментальных данных, и являются фактическим стандартом при разработке шасси и систем активной безопасности у автопроизводителей. Их недостатками применительно к рассматриваемой задаче являются проприетарность, высокая стоимость лицензий и ориентация на стендовое и полунатурное моделирование, а не на интеграцию в интерактивные приложения с жёстким бюджетом кадра. Перенос таких комплексов в игровой движок затруднителен и, как правило, экономически неоправдан.

Группу игровых решений образуют физические подсистемы игровых движков и коммерческие дополнения к ним. Наиболее показательным примером является дополнение Vehicle Physics Pro для движка Unity — самостоятельный физический пакет, реализующий реалистичную модель покрышки на основе формулы Пасейки. Он обеспечивает высокое качество симуляции, однако распространяется с закрытым исходным кодом, является платным и жёстко привязан к движку Unity, что делает невозможным его использование в других средах. Другим примером служит подсистема NVIDIA PhysX Vehicle, входящая в состав ряда движков; она бесплатна и кроссплатформенна, но использует упрощённую модель покрышки, уступающую по реализму полноценной модели Пасейки. Общим недостатком

игровых решений является привязка к конкретному движку или упрощённость модели.

Проекты с открытым исходным кодом представлены, в частности, библиотекой Project Chrono, включающей модуль моделирования транспортных средств с поддержкой моделей покрышки, в том числе модели Пасейки. Такие проекты открыты, научно обоснованы и кроссплатформенны, однако ориентированы на точное многотельное моделирование и не предназначены для работы в реальном времени без существенной адаптации; их интеграция в игровой движок сопряжена со значительными трудностями.

Сопоставление перечисленных групп решений показывает, что ни одна из них в полной мере не отвечает требованиям лёгкого, открытого, движко-независимого модуля реального времени с реалистичной моделью покрышки. Профессиональные комплексы точны, но закрыты и тяжеловесны; игровые решения доступны, но привязаны к движку или упрощены; открытые проекты открыты, но не оптимизированы для реального времени. Именно эта незанятая ниша определяет позиционирование разрабатываемого модуля и обуславливает новизну предлагаемого решения. Качественное сопоставление разрабатываемого модуля с указанными решениями по ряду верифицируемых признаков приведено в таблице 1.1.

Таблица 1.1 — Сопоставление программных решений автомобильной физики

Признак	VPP	PhysX	Chrono	Модуль
Открытый код	нет	частично	да	да
Модель Пасейки	да	нет	да	да
Независимость от движка	нет	частично	да	да
Кроссплатформенность	огранич.	да	да	да
Реальное время	да	да	нет	да

1.7 Выбор модели и выводы по главе

Сравнение рассмотренных моделей удобно представить в виде сводной таблицы, в которой каждая модель оценивается по ключевым критериям:

точности воспроизведения характеристик, вычислительной сложности, способности описывать комбинированное нагружение и требованиям к исходным данным (таблица 1.2).

Таблица 1.2 — Сравнение математических моделей покрышки

Критерий	Пасейка	Щёточная	Фиала	Дугофф
Точность	высокая	средняя	средняя	средняя
Вычисл. сложность	низкая	средняя	низкая	низкая
Комбинир. нагружение	да	да	частично	да
Физ. интерпретируемость	нет	высокая	средняя	средняя
Требования к данным	высокие	низкие	низкие	низкие

В настоящей работе для реализации в составе разрабатываемого модуля выбрана модель Пасейки в её базовой форме. Такой выбор обусловлен совокупностью факторов. Во-первых, модель Пасейки обеспечивает наилучшую точность воспроизведения нелинейной характеристики покрышки среди рассмотренных моделей, что критично для достоверной симуляции поведения автомобиля на пределе сцепления. Во-вторых, вычисление силы по формуле (1.1) сводится к одному вызову обратного тангенса и нескольким арифметическим операциям, что делает модель чрезвычайно экономичной и пригодной для работы на высоких частотах обновления. В-третьих, базовая форма модели требует задания лишь четырёх коэффициентов на каждое направление, которые могут быть подобраны эмпирически без дорогостоящих стендовых испытаний, что снимает основное ограничение полной версии модели. Комбинированное нагружение учитывается отдельно — посредством ограничения результирующей силы эллипсом трения, что будет описано во второй главе.

Таким образом, в первой главе сформулированы требования к модели покрышки для симуляторов реального времени, рассмотрены четыре основные полуэмпирические и аналитические модели, проведён их

сравнительный анализ и обоснован выбор модели Пасейки в качестве основы разрабатываемого модуля.

2 Математические модели узлов автомобиля

В данной главе описываются математические модели, положенные в основу разрабатываемого модуля. Модуль реализует физику автомобиля как совокупность взаимодействующих подмоделей: модели покрышки, независимой подвески, рулевого управления, дифференциала, двигателя и коробки передач. Такая декомпозиция отражает реальную структуру трансмиссии и ходовой части автомобиля и позволяет разрабатывать, отлаживать и заменять отдельные узлы независимо друг от друга.

Прежде чем переходить к описанию отдельных моделей, целесообразно представить общую расчётную схему, связывающую их в единое целое. На каждом шаге симуляции расчёт выполняется в определённом порядке, отражающем причинно-следственные связи между узлами. Сначала модель двигателя по текущим оборотам и положению дроссельной заслонки вычисляет крутящий момент; этот момент через сцепление и коробку передач передаётся на дифференциал, который распределяет его между ведущими колёсами. Далее для каждого колеса определяется контакт с поверхностью, рассчитываются сила подвески и нормальная нагрузка, после чего по кинематическим параметрам вычисляются продольное скольжение и угол увода. На основе скольжения и нормальной нагрузки модель покрышки рассчитывает продольную и боковую силы с учётом эллипса трения. Реактивный момент покрышки воздействует на угловую скорость колеса, замыкая контур «двигатель — трансмиссия — колесо — покрышка», а силы, приложенные в точках контакта, через подвеску определяют движение кузова. Описанный порядок вычислений важен для корректности результата и явно зафиксирован в программном интерфейсе модуля.

Расчёт сил покрышки выполняется в локальной системе координат колеса, оси которой связаны с направлением его качения, поперечным направлением и нормалью к поверхности. Линейная скорость точки контакта, получаемая из состояния твёрдого тела, преобразуется в эту локальную

систему, что позволяет единообразно вычислять продольную и поперечную составляющие скорости независимо от ориентации автомобиля в пространстве. Рассчитанные в локальной системе силы затем проецируются на плоскость дороги, заданную нормалью в точке контакта, и преобразуются обратно в глобальную систему координат для применения к твёрдому телу. Такое разделение систем координат обеспечивает корректную работу модели при движении по наклонным и неровным поверхностям, когда плоскость качения колеса не совпадает с горизонтальной плоскостью.

2.1 Модель покрышки и боковая сила трения

Расчёт боковой силы трения покрышки является ключевым элементом реалистичности симуляции. Исходной кинематической величиной служит угол увода (угол скольжения) колеса, определяемый как угол между плоскостью вращения колеса и направлением его фактического движения. Угол увода вычисляется по составляющим линейной скорости колеса в его системе координат:

$$\alpha = \arctan(-v_x / (|v_z| + \varepsilon)) \quad (2.1)$$

где v_x и v_z — поперечная и продольная составляющие линейной скорости колеса соответственно, а ε — малое пороговое значение, предотвращающее деление на ноль при малых скоростях движения. Аналогичным образом для продольного направления вводится коэффициент продольного скольжения, определяемый как относительная разность между окружной скоростью колеса и скоростью его поступательного движения:

$$\kappa = (\omega \cdot R - v_z) / \max(|v_z|, v_{\text{floor}}) \quad (2.2)$$

где ω — угловая скорость колеса, R — радиус колеса, а v_{floor} — нижний порог скорости, ограничивающий рост скольжения вблизи состояния покоя. Полученные значения κ и α подаются на вход формулы Пасейки (1.1) для расчёта продольной и боковой сил соответственно.

Однако при прямом использовании статического угла увода по формуле (2.1) обнаруживается характерный недостаток: автомобиль нереалистично скатывается с наклонных поверхностей, поскольку боковая сила трения возникает мгновенно и не учитывает конечного времени её нарастания. В действительности реакция покрышки на изменение угла увода запаздывает — деформация протектора и, соответственно, боковая сила нарастают по мере прохождения покрышкой некоторого пути, называемого длиной релаксации. Для устранения указанного недостатка в работе введён динамический угол скольжения, моделирующий конечное время нарастания силы трения:

$$\text{relax} = \text{clamp}(|v_z| \cdot \Delta t / L_{\text{rel}}, 0, 1) \quad (2.3)$$

$$\alpha_d \leftarrow \alpha_d + (\alpha - \alpha_d) \cdot \text{relax} \quad (2.4)$$

где L_{rel} — длина релаксации покрышки, Δt — шаг симуляции, α_d — динамический угол скольжения. Выражение (2.4) представляет собой релаксационное сглаживание: на каждом шаге динамический угол приближается к мгновенному значению угла увода тем быстрее, чем больший путь покрышка проходит за шаг. При большой скорости движения сила трения нарастает практически мгновенно, тогда как при малой скорости или на наклонной поверхности нарастание происходит постепенно, что и обеспечивает физически достоверное поведение. Боковое ускорение, реализуемое покрышкой, связывается с динамическим углом скольжения соотношением:

$$a_{\text{lat}} = (|v|^2 / R) \cdot \tan(\alpha_d) \quad (2.5)$$

Параметры L_{rel} и α_{peak} позволяют настраивать характеристики покрышки: длина релаксации определяет время нарастания силы трения, а предельный угол α_{peak} задаёт порог насыщения, при котором достигается максимум боковой силы. Введение динамического угла скольжения является одним из основных результатов работы, поскольку именно оно обеспечивает корректную симуляцию динамического перехода боковой силы и устраняет нефизичное скатывание автомобиля с уклонов.

Окончательная сила, реализуемая покрышкой, формируется с учётом комбинированного нагружения. Продольная и боковая силы, рассчитанные по формуле Пасейки независимо, нормируются на свои пиковые значения, после чего проверяется условие принадлежности результирующего вектора эллипсу трения. Если суммарная нормированная сила превышает единицу, обе составляющие масштабируются обратно пропорционально модулю результирующего вектора. Тем самым гарантируется, что покрышка не может реализовать силу, превышающую предел сцепления, при одновременном продольном и боковом скольжении, что соответствует физической природе процесса.

Геометрически условие эллипса трения означает, что конец вектора суммарной силы, отнесённого к пиковым значениям по каждому направлению, не может выходить за пределы единичной окружности. Формально, если обозначить нормированные составляющие как $g_x = F_x / D_{long}$ и $g_y = F_y / D_{lat}$, то их совместная величина $g = \sqrt{g_x^2 + g_y^2}$ не должна превышать единицы; при $g > 1$ обе составляющие умножаются на $1/g$. Эта же величина g удобно интерпретируется как мера интенсивности проскальзывания покрышки: при g , близком к нулю, покрышка работает в линейной области, а при g , достигающем единицы, она выходит на предел сцепления и начинает скользить. В реализации нормированная величина g используется дополнительно как показатель степени проскальзывания, который может применяться, например, для управления визуальными и звуковыми эффектами заноса.

2.2 Независимая подвеска

Для каждого колеса реализована независимая подвеска типа МакФерсон. Сила подвески рассчитывается как сумма упругой и демпфирующей составляющих. Упругая составляющая пропорциональна отклонению текущей длины пружины от её длины в состоянии покоя, а

демпфирующая составляющая пропорциональна скорости изменения длины пружины:

$$F_spring = (L_0 - L) \cdot k_s \quad (2.6)$$

$$F_damper = ((L_prev - L) / \Delta t) \cdot k_d \quad (2.7)$$

$$F_susp = \max(0, F_spring + F_damper) \quad (2.8)$$

где L_0 — длина пружины в состоянии покоя, L — текущая длина пружины, k_s — жёсткость пружины, k_d — коэффициент демпфирования, L_prev — длина пружины на предыдущем шаге. Текущая длина подвески определяется как расстояние от точки крепления до точки контакта колеса с поверхностью, найденной методом рейкаста, с учётом радиуса колеса. Ограничение снизу нулём в выражении (2.8) предотвращает появление отрицательных (тянущих) сил при полном растяжении подвески: реальная пружина в подвеске не может притягивать колесо к кузову при отрыве от поверхности. Рассчитанная сила подвески определяет нормальную нагрузку на покрышку и, следовательно, через коэффициент D формулы Пасейки, влияет на максимальные силы, которые способна реализовать покрышка.

Определение контакта колеса с поверхностью выполняется методом трассировки луча (рейкаста), направленного вдоль оси подвески. Результатом рейкаста являются точка контакта, нормаль к поверхности в этой точке и расстояние до неё. Использование рейкаста вместо моделирования объёмного колеса является общепринятым упрощением, оправданным в задачах реального времени: оно существенно снижает вычислительные затраты и при этом обеспечивает достаточную для большинства приложений достоверность. Найденная нормаль к поверхности используется при проецировании сил покрышки на плоскость дороги, что обеспечивает корректную работу модели на наклонных и неровных поверхностях. Связь подвески и покрышки через нормальную нагрузку является ключевой: при сжатии подвески на неровностях нормальная нагрузка возрастает, увеличивая сцепление, а при

разгрузке — снижается вплоть до полного отрыва колеса, когда силы покрышки обращаются в ноль.

2.3 Рулевое управление по принципу Аккермана

При повороте автомобиля внутреннее и внешнее по отношению к центру поворота колёса движутся по дугам разного радиуса и потому должны поворачиваться на разные углы для минимизации бокового проскальзывания. Корректные углы поворота управляемых колёс определяются из геометрии Аккермана:

$$\theta_i = \arctan(L_w / (R - W/2)) \quad (2.9)$$

$$\theta_o = \arctan(L_w / (R + W/2)) \quad (2.10)$$

где L_w — колёсная база, W — ширина колеи, θ_i и θ_o — углы поворота внутреннего и внешнего колёс соответственно, R — радиус поворота. В реализации углы Аккермана вычисляются для левого и правого колёс в зависимости от направления поворота и масштабируются коэффициентом усилия рулевого управления. Дополнительно вводится корректировка углов поворота на основе бокового ускорения колёс: при возникновении значительного бокового ускорения углы поворота уменьшаются, что моделирует эффект самовозврата рулевого управления и стабилизирует поведение автомобиля. Корректирующее воздействие ограничивается заданным максимальным углом, а итоговые углы поворота сглаживаются во времени, обеспечивая плавность работы рулевого управления.

2.4 Дифференциал

В модуле реализованы два типа дифференциала — открытый и заблокированный. При открытом дифференциале входной крутящий момент распределяется поровну между выходными валами:

$$T_{out} = T_{in} \cdot r_d \cdot 0.5 \quad (2.11)$$

Однако при таком распределении не учитывается разница угловых скоростей колёс, возникающая, например, в повороте или при попадании одного из колёс на скользкую поверхность. Поэтому при работе открытого дифференциала вводится корректирующая величина, перераспределяющая момент в зависимости от разности угловых скоростей левого и правого колёс:

$$\Delta T = ((\omega_L - \omega_R) \cdot 0.5 / \Delta t) \cdot I_w \quad (2.12)$$

$$T_L = T_{in} \cdot 0.5 \cdot r_d - \Delta T, \quad T_R = T_{in} \cdot 0.5 \cdot r_d + \Delta T \quad (2.13)$$

где T_{in} — входной крутящий момент, r_d — передаточное число (коэффициент) дифференциала, ω_L и ω_R — угловые скорости левого и правого колёс, I_w — момент инерции колеса. При заблокированном дифференциале корректирующая величина не вводится, и момент распределяется между колёсами поровну, что обеспечивает их синхронное вращение. Принятая архитектура позволяет добавлять иные типы блокировки, например дифференциал повышенного трения с вискомуфтой, без изменения остальных компонентов модели.

2.5 Двигатель и коробка передач

Крутящий момент двигателя вычисляется на основе табличной зависимости момента от частоты вращения коленчатого вала с учётом внутреннего трения и инерционности. Максимальный эффективный момент определяется по кривой момента, после чего из него вычитается момент внутреннего трения, линейно зависящий от оборотов, и учитывается положение дроссельной заслонки:

$$T_{max} = T_{curve}(n) \cdot m_{eng} \quad (2.14)$$

$$T_{fric} = n \cdot f_{coeff} + f_{start} \quad (2.15)$$

$$T_{eng} = T_{max} \cdot throttle - T_{fric} \quad (2.16)$$

$$\varepsilon_{eng} = (T_{eng} - T_{clutch}) / I_{eng} \quad (2.17)$$

где $T_curve(n)$ — табличная зависимость момента от оборотов n , m_eng — множитель мощности, f_coeff и f_start — коэффициенты внутреннего трения, $throttle$ — положение дроссельной заслонки (от 0 до 1), T_clutch — момент сцепления, I_eng — момент инерции двигателя, ε_eng — угловое ускорение коленчатого вала. Угловая скорость двигателя интегрируется по полученному ускорению и ограничивается диапазоном между оборотами холостого хода и максимальными оборотами.

Использование табличной зависимости момента от оборотов вместо аналитической формулы позволяет точно задавать характеристику конкретного двигателя, включая характерные особенности — провалы и пики момента, форму кривой в зоне максимальной мощности. Промежуточные значения момента между узлами таблицы вычисляются интерполяцией. Член внутреннего трения, линейно зависящий от оборотов, моделирует насосные и механические потери, возрастающие с частотой вращения, а постоянная составляющая трения определяет момент, необходимый для поддержания холостого хода. Совокупность этих составляющих обеспечивает реалистичную динамику разгона и торможения двигателем, а также корректное поведение на холостом ходу.

Передача момента от двигателя к колёсам осуществляется через сцепление и коробку передач. Сцепление моделируется как муфта с ограниченной передаваемой ёмкостью: передаваемый момент пропорционален разности угловых скоростей коленчатого вала и первичного вала коробки и ограничивается максимальной ёмкостью сцепления, при этом степень замыкания сцепления плавно изменяется в зависимости от оборотов двигателя. Коробка передач пересчитывает угловую скорость и крутящий момент через передаточное число текущей передачи:

$$\omega_out = \omega_in / r_i, \quad T_out = T_in \cdot r_i \quad (2.18)$$

где r_i — передаточное число текущей передачи. Система настройки коробки передач позволяет задавать произвольное количество передач и

соответствующие им передаточные числа, а также время переключения, в течение которого коробка переходит в нейтральное положение. Переключение передач реализовано с конечным временем срабатывания, что моделирует разрыв потока мощности при смене передачи.

Модель сцепления требует особого внимания, поскольку именно она связывает вращающийся с собственной инерцией двигатель с трансмиссией и колёсами. При полностью разомкнутом сцеплении двигатель раскручивается свободно, а колёса вращаются независимо; при полностью замкнутом сцеплении двигатель и трансмиссия образуют единую вращающуюся систему. Промежуточные состояния — частичное замыкание при трогании с места — описываются плавным изменением степени замыкания в зависимости от оборотов: при оборотах ниже определённого порога сцепление разомкнуто, выше другого порога — замкнуто, а между ними степень замыкания изменяется монотонно. Передаваемый момент дополнительно сглаживается во времени демпфирующим коэффициентом, что предотвращает скачкообразные изменения момента и связанные с ними колебания. Такая модель, не претендуя на детальное воспроизведение термодинамики фрикционного диска, корректно отражает функциональное поведение сцепления и обеспечивает плавное трогание и переключение передач.

Совместная работа двигателя, сцепления и коробки передач образует замкнутый контур: обороты двигателя определяют передаваемый сцеплением момент, момент через коробку и дифференциал поступает на колёса, а нагрузка со стороны колёс через ту же цепочку влияет на обороты двигателя. Корректное согласование инерций и моментов в этом контуре существенно для устойчивости и достоверности симуляции, особенно в переходных режимах — при трогании, переключении передач и резком сбросе газа.

Таким образом, во второй главе описаны математические модели всех узлов автомобиля, реализованных в модуле. Совокупность этих моделей образует замкнутую расчётную схему: момент двигателя через сцепление, коробку передач и дифференциал передаётся на колёса, угловая скорость

которых совместно со скоростью кузова определяет скольжение покрышки, а рассчитанные по модели Пасейки силы прикладываются в точках контакта и через подвеску воздействуют на кузов автомобиля.

3 Архитектура и программная реализация модуля

3.1 Общая архитектура библиотеки

Модуль реализован в виде динамически подключаемой библиотеки (DLL) на языке C++ и предоставляет внешний программный интерфейс (API) для инициализации модели, передачи входных параметров и получения результирующих сил. На каждом шаге симуляции вызывающий код передаёт в библиотеку текущее состояние автомобиля — скорости, положение, параметры поверхности и команды управления, — а библиотека возвращает векторы импульсов сил, которые необходимо приложить в точках контакта колёс с дорогой. Тем самым библиотека берёт на себя всю физическую логику автомобиля, оставляя вызывающей среде только задачи интегрирования движения твёрдого тела, отрисовки и обработки ввода.

Внутренняя архитектура построена по модульному принципу: каждый физический узел — покрышка, подвеска, двигатель, коробка передач, дифференциал — реализован как независимый компонент. Для каждого колеса создан набор подмоделей, отвечающих за отдельные аспекты физики: расчёт углового ускорения колеса, определение контакта с поверхностью методом рейкаста, вычисление сил трения покрышки, расчёт состояния пружины подвески, определение скольжения и поворот колеса. Такая декомпозиция позволяет заменять или расширять отдельные модели без влияния на остальные части системы. Например, замена модели покрышки Пасейки на модель Дугоффа затрагивает только соответствующий компонент и не требует изменения моделей подвески или трансмиссии.

Принятая модульная организация подтвердила свою практическую ценность в ходе разработки. Первые результаты тестовой интеграции выявили проблемы с моделью бокового проскальзывания колеса, однако благодаря несвязности компонентов системы эти проблемы были локализованы в одном модуле и быстро устранены без влияния на остальные узлы. Это иллюстрирует

одно из ключевых преимуществ выбранной архитектуры — возможность независимой отладки и сопровождения отдельных компонентов.

3.2 Программный интерфейс на основе C-ABI

Ключевым архитектурным решением, обеспечивающим независимость модуля от конкретной среды исполнения, является построение программного интерфейса на основе чистого двоичного интерфейса языка C (C-ABI). В отличие от интерфейса C++, двоичный интерфейс которого не стандартизован и различается между компиляторами, интерфейс языка C обладает стабильным и предсказуемым двоичным представлением, поддерживаемым всеми распространёнными языками и средами через механизмы внешних вызовов — P/Invoke в платформе .NET, механизм FFI в различных языках и аналогичные средства. Это позволяет вызывать функции библиотеки из произвольной среды — игрового движка, научного пакета или собственного приложения — без перекомпиляции и без привязки к конкретному компилятору.

Внешний интерфейс библиотеки объявлен в блоке `extern "C"`, что отключает декорирование имён, свойственное C++, и обеспечивает экспорт функций под предсказуемыми именами. Интерфейс включает функции создания и уничтожения объекта симуляции автомобиля, а также функции обновления состояния трансмиссии и колёс. Объект симуляции представлен непрозрачным дескриптором (`handle`), что скрывает внутреннее устройство модели от вызывающей стороны и обеспечивает её инкапсуляцию. Состояние автомобиля передаётся через структуры данных явно, на каждом вызове, что исключает использование глобального состояния и обеспечивает потокобезопасность: несколько экземпляров симуляции могут работать одновременно в разных потоках без взаимного влияния.

Характерной особенностью интерфейса является то, что обновление состояния трансмиссии и обновление состояния колёс всегда выполняются совместно, парным вызовом на каждом шаге физики. Это отражает причинно-

следственную связь между подмоделями: момент, рассчитанный трансмиссией, прикладывается к колёсам, а угловые скорости колёс, в свою очередь, влияют на работу дифференциала и сцепления на следующем шаге. Подобная организация интерфейса делает порядок вычислений явным и предотвращает рассогласование состояния подмоделей.

Передача данных через границу библиотеки организована при помощи структур данных с фиксированной раскладкой полей, согласованной между библиотекой и вызывающей стороной. Вызывающая сторона заполняет входную структуру текущим состоянием — линейными и угловыми скоростями, положениями и нормальными векторами точек контакта колёс, командами управления — и получает выходную структуру с импульсами сил для каждого колеса и вспомогательными величинами, такими как углы поворота колёс для визуализации. Передача состояния «по значению», без хранения ссылок на объекты вызывающей среды, обеспечивает чёткую границу между библиотекой и средой и исключает зависимость от особенностей управления памятью конкретного движка. Поскольку библиотека не выполняет динамического выделения памяти на горячем пути расчёта и не обращается к глобальным переменным, она пригодна для одновременного использования в нескольких независимых экземплярах.

3.3 Численное интегрирование с фиксированным шагом

Физическая модель интегрируется с фиксированным шагом по времени. Выбор фиксированного, а не переменного шага обусловлен двумя соображениями. Во-первых, фиксированный шаг обеспечивает детерминизм вычислений: при одинаковых входных данных результат симуляции воспроизводится в точности, что важно как для отладки, так и для возможного регрессионного тестирования физики и сетевой синхронизации. Во-вторых, фиксированный шаг согласуется с принятой в большинстве игровых движков организацией физического цикла, в котором физика обновляется с постоянной частотой независимо от частоты отрисовки кадров.

При реализации численного интегрирования особое внимание потребовалось уделить устойчивости расчёта угловой скорости колеса. Продольная сила покрышки, рассчитываемая по формуле Пасейки, обладает очень высокой жёсткостью по отношению к скорости вращения колеса: наклон характеристики в начале координат, определяемый произведением $V \cdot C \cdot D$, при малых скоростях движения дополнительно возрастает за счёт деления на скорость. Явное интегрирование такой жёсткой зависимости при фиксированном шаге приводит к колебаниям угловой скорости колеса и расходимости расчёта — колесо либо начинает осциллировать, либо неограниченно раскручивается до предельного значения.

Для обеспечения безусловной устойчивости в работе применена неявная схема линеаризации реакции покрышки относительно текущей угловой скорости колеса. Зависимость продольной силы от скольжения линеаризуется в окрестности текущей рабочей точки, после чего приращение угловой скорости находится из неявного соотношения, учитывающего наклон характеристики:

$$\Delta\omega = (T_drive - T_react) / (I_w/\Delta t + s) \quad (3.1)$$

где T_drive — приводной момент, T_react — реактивный момент покрышки, вычисленный на предыдущем шаге, I_w — момент инерции колеса, а s — приведённый к угловой скорости наклон характеристики покрышки, равный произведению наклона силы по скольжению на квадрат радиуса колеса, делённому на ограниченную снизу скорость движения. Такая схема является безусловно устойчивой и позволяет колесу плавно выходить на установившуюся скорость качения вместо раскрутки до ограничителя. Нижний порог скорости в знаменателе сохраняет конечность наклона характеристики вблизи состояния покоя.

Торможение реализовано отдельно от приводного момента и всегда направлено против текущего вращения колеса. Приращение угловой скорости от тормозного момента ограничивается так, чтобы торможение могло за один

шаг лишь остановить колесо, но не обратить направление его вращения вспять, что соответствует физической природе тормозного механизма. Итоговая угловая скорость колеса ограничивается заданным диапазоном.

Аналогичный приём ограничения применён и при расчёте скольжения покрышки: для предотвращения сингулярности коэффициента продольного скольжения и угла увода при скоростях, близких к нулю, в знаменатель соответствующих выражений введён нижний порог скорости. Это обеспечивает корректное поведение модели на всём диапазоне скоростей, включая трогание с места и полную остановку.

Выбор неявной схемы для расчёта продольной динамики колеса представляет собой методически важное решение, поэтому остановимся на нём подробнее. Уравнение вращения колеса связывает изменение его угловой скорости с разностью приводного и реактивного моментов. Реактивный момент покрышки зависит от продольного скольжения, которое, в свою очередь, зависит от угловой скорости колеса. Вблизи режима качения без проскальзывания эта зависимость очень крутая: малое изменение угловой скорости вызывает большое изменение силы. При явном интегрировании, когда сила вычисляется по состоянию на предыдущем шаге, такая крутизна приводит к тому, что система «перескакивает» равновесное состояние и начинает колебаться с нарастающей амплитудой. Уменьшение шага интегрирования смягчает проблему, но не устраняет её и противоречит требованию производительности. Неявная схема, в которой сила линеаризуется относительно искомой новой угловой скорости, лишена этого недостатка: она безусловно устойчива при любом шаге и обеспечивает плавный выход колеса на равновесную скорость качения. Именно поэтому в модуле принята неявная линеаризованная схема, описываемая соотношением (3.1).

3.4 Структура программного кода

Программная реализация организована в виде набора компонентов, объединённых в пространстве имён модуля. Каждому физическому узлу соответствует отдельный класс с методами инициализации и обновления. Класс модели двигателя рассчитывает крутящий момент и угловое ускорение коленчатого вала; класс коробки передач управляет переключением передач и пересчётом момента; класс сцепления вычисляет передаваемый момент; класс дифференциала распределяет момент между колёсами; классы тормозов и рулевого управления рассчитывают тормозные моменты и углы поворота колёс соответственно. Для каждого колеса предусмотрены отдельные компоненты, отвечающие за подвеску, расчёт скольжения, продольную динамику, силу покрышки и визуальное представление.

Декомпозиция по колесу заслуживает отдельного пояснения, поскольку именно на уровне колеса сосредоточена основная физическая логика. Для каждого из колёс создаётся набор подмоделей: подмодель подвески рассчитывает силу упругого элемента и нормальную нагрузку на основе текущей и предыдущей длин пружины; подмодель скольжения по линейной скорости колеса и его угловой скорости определяет продольное скольжение и динамический угол увода с учётом релаксации; подмодель продольной динамики по неявной схеме обновляет угловую скорость колеса под действием приводного и тормозного моментов и реакции покрышки; подмодель покрышки по скольжению и нормальной нагрузке вычисляет продольную и боковую силы с учётом эллипса трения и возвращает результирующий вектор силы, спроецированный на плоскость дороги; наконец, подмодель визуального представления рассчитывает углы вращения и поворота колеса для последующей отрисовки. Такое детальное разбиение позволяет точно локализовать любую неисправность и независимо настраивать отдельные аспекты поведения колеса.

Соглашение об индексации колёс зафиксировано в коде: переднее левое, переднее правое, заднее левое и заднее правое колёса имеют фиксированные

индексы, что используется, в частности, при распределении тормозного момента по осям и при работе дифференциала с задними колёсами. Параметры покрышки, такие как коэффициент жёсткости формулы Пасейки, вычисляются однократно при инициализации исходя из заданных пикового значения и угла его достижения, что исключает повторные вычисления на горячем пути и повышает производительность.

Такое разделение ответственности между классами обеспечивает соответствие программной структуры физической структуре моделируемого объекта, упрощает понимание и сопровождение кода и позволяет тестировать отдельные модели изолированно. Параметры каждого узла передаются через соответствующие структуры конфигурации при инициализации, что отделяет настройку модели от логики расчёта и позволяет хранить параметры различных автомобилей во внешних конфигурационных данных.

3.5 Кросс-платформенная сборка модуля

Одним из определяющих свойств разрабатываемого модуля является его кросс-платформенность. Поскольку модуль написан на стандартном языке C++ и не использует платформенно-зависимых средств помимо стандартной библиотеки, его исходный код может быть скомпилирован под любую целевую платформу, для которой существует соответствующий компилятор. На платформе Windows модуль собирается в виде динамически подключаемой библиотеки с расширением `.dll`, на платформах семейства Linux — в виде разделяемого объекта `.so`, а на игровых консолях — в виде платформенно-специфичного динамического модуля. Во всех случаях исходный код модуля остаётся неизменным, а различается лишь используемый набор инструментов разработки целевой платформы.

Такая организация принципиально отличает предлагаемый модуль от решений, привязанных к конкретному движку или среде исполнения. Для переноса модуля на новую платформу разработчику достаточно скомпилировать его исходный код соответствующим инструментарием — не

требуется ни переписывания физической логики, ни адаптации к особенностям конкретного движка. Это существенно снижает трудозатраты на поддержку приложения, ориентированного на несколько платформ, и устраняет дублирование физической модели, неизбежное при использовании движко-зависимых решений.

Отделение физической модели от среды исполнения через стабильный двоичный интерфейс обеспечивает и долгосрочную сопровождаемость: обновление или исправление физической модели сводится к замене одного бинарного файла без изменения вызывающего приложения, а сам модуль может развиваться независимо от движков, в которых он используется. Тем самым достигается одна из основных целей работы — создание переносимого, самодостаточного компонента автомобильной физики.

Таким образом, в третьей главе описаны общая модульная архитектура библиотеки, построение программного интерфейса на основе стабильного двоичного интерфейса языка C, особенности численного интегрирования с фиксированным шагом и применённая для обеспечения устойчивости неявная схема расчёта угловой скорости колеса, структура программного кода, а также организация кросс-платформенной сборки модуля.

4 Интеграция в игровой движок и результаты тестирования

4.1 Интеграция в Unity

Разработка и тестовая интеграция библиотеки выполнялись в среде игрового движка Unity. Выбор Unity в качестве тестовой платформы обусловлен его широкой распространённостью, наличием встроенной физической подсистемы для интегрирования движения твёрдого тела и поддержкой вызова функций неуправляемых библиотек через механизм P/Invoke. Важно подчеркнуть, что интеграция именно в Unity носит иллюстративный характер: благодаря построению интерфейса на основе C-ABI тот же бинарный модуль может быть подключён к любому другому движку или приложению без изменений в его коде.

Взаимодействие движка и библиотеки организовано следующим образом. На каждом физическом шаге вызывающий код собирает необходимую информацию о состоянии автомобиля — положение, скорости, параметры конфигурации автомобиля, результаты определения контакта колёс с поверхностью — и передаёт её в библиотеку. В пределах того же физического кадра библиотека выполняет расчёт и возвращает импульсы сил для каждого колеса, которые затем прикладываются движком в точках соприкосновения покрышек с поверхностью. Интегрирование движения кузова автомобиля как твёрдого тела выполняет физическая подсистема движка, тогда как все силы, определяющие это движение, рассчитываются библиотекой.

Тестирование проводилось на специально подготовленном виртуальном полигоне, включавшем разнообразные статические и динамические физические объекты: участки дорог с различными типами покрытия, стены, наклонные поверхности и зоны, имитирующие бездорожье. Такой набор условий позволил проверить поведение модели в широком диапазоне ситуаций — от движения по ровной дороге до преодоления уклонов и

неровностей, где особенно важна корректная работа подвески и динамической модели бокового скольжения.

Со стороны вызывающего кода интеграция потребовала описания соответствия структур данных библиотеки и объявления импортируемых функций с указанием соглашения о вызове. После однократного описания этого соответствия дальнейшее взаимодействие сводится к заполнению входных структур на каждом физическом шаге и применению возвращаемых импульсов к твёрдому телу автомобиля средствами движка. Объект симуляции создаётся при инициализации автомобиля и уничтожается при его удалении, а его дескриптор хранится на стороне вызывающего кода. Подобная схема интеграции типична для подключения неуправляемых библиотек и не зависит от специфики конкретного движка, что подтверждает переносимость предложенного решения.

4.2 Оценка производительности

Оценка производительности проводилась для одного автомобиля при расчётах в одном потоке процессора AMD Ryzen 7 7700. За эталонный бюджет кадра принималось время 16 мс, соответствующее частоте отрисовки около 60 кадров в секунду — типичной для интерактивных приложений. В этот бюджет в реальном приложении входят обработка ввода, графика, игровая логика и физика, поэтому доля, занимаемая расчётом автомобильной физики, должна оставаться малой. Физическая модель обновлялась с фиксированным шагом, а измерения проводились при различных частотах обновления — от 100 до 20000 Гц. Результаты измерений приведены в таблице 4.1.

Таблица 4.1 — Затраты времени на расчёт физики при различных частотах обновления

Частота, Гц	Шаг, мс	Время расчёта, мс	Доля кадра, %
100	10	≈ 0,005	0,03
1 000	1	≈ 0,018	0,11
10 000	0,1	≈ 0,22	1,3
20 000	0,05	≈ 0,56	3,5

Как видно из таблицы, при частоте обновления 100 Гц — вдвое превышающей стандартную для большинства игровых движков частоту 50–60 Гц — расчёт одного физического шага с учётом нагрузки самого движка занимает порядка 0,005 мс, что составляет лишь сотые доли процента бюджета кадра. Даже при частоте 1000 Гц затраты не превышают приблизительно 0,11 % бюджета кадра, что свидетельствует о высокой степени оптимизации модели. С ростом частоты обновления затраты растут практически линейно, оставаясь умеренными вплоть до экстремальных частот в десятки тысяч герц.

Полученные результаты подтверждают, что разработанная модель обладает значительным запасом производительности при типичных рабочих частотах. Этот запас является ресурсом для последующего масштабирования: при необходимости одновременной симуляции двух и более автомобилей, в том числе управляемых искусственным интеллектом, освободившийся бюджет может быть использован для параллельных расчётов. Вместе с тем для эффективной симуляции большого числа автомобилей потребуются дополнительные оптимизации и распараллеливание вычислений, что выходит за рамки настоящей работы.

Линейный характер роста затрат с увеличением частоты обновления заслуживает отдельного комментария. Поскольку объём вычислений на одном шаге постоянен, а число шагов в единицу времени растёт пропорционально частоте, суммарные затраты в единицу времени должны возрастать линейно, что и наблюдается в измерениях. Это означает, что выбор частоты обновления представляет собой прямой компромисс между точностью интегрирования и вычислительными затратами: повышение частоты улучшает устойчивость и точность численной схемы, но пропорционально увеличивает нагрузку. Полученная количественная зависимость позволяет осознанно выбирать рабочую частоту исходя из доступного бюджета кадра и числа одновременно моделируемых автомобилей. Для одиночного автомобиля практически целесообразной представляется частота порядка 1000 Гц, обеспечивающая

высокую устойчивость при затратах около одной десятой процента бюджета кадра.

Следует подчеркнуть, что приведённые значения получены с учётом полной нагрузки игрового движка, а не в изоляции, и потому отражают реальные эксплуатационные затраты, а не теоретический минимум. Измерения проводились в одном потоке процессора, без использования векторных инструкций и многопоточности, что оставляет дополнительные резервы для оптимизации. Сравнение полученных показателей с производительностью конкурирующих решений — таких как Vehicle Physics Pro и NVIDIA PhysX Vehicle — не проводилось в рамках настоящей работы, поскольку корректное сравнение требует проведения замеров всех решений на едином стенде по единой методике; такое сравнение отнесено к направлениям дальнейшей работы.

4.3 Оценка корректности и валидация модели

Корректность работы физической модели оценивалась качественно, по соответствию поведения виртуального автомобиля ожидаемому поведению реального транспортного средства в характерных манёврах. Проверялись разгон с пробуксовкой ведущих колёс, торможение, в том числе торможение в повороте, прохождение поворотов с заносом и без, движение по наклонным поверхностям и преодоление неровностей. Во всех рассмотренных ситуациях поведение модели соответствовало физически ожидаемому: автомобиль реагировал на управляющие воздействия предсказуемо, силы реакции покрышек формировались корректно, а введение динамического угла скольжения устранило ранее наблюдавшееся нефизичное скатывание с уклонов.

Особое внимание уделялось проверке поведения, ради корректности которого был введён динамический угол скольжения. При неподвижном или медленно движущемся автомобиле на наклонной поверхности статическая формула угла увода даёт мгновенную боковую силу, которая нефизично

удерживает или, напротив, сбрасывает автомобиль с уклона рывком. После введения релаксационного сглаживания угла увода поведение стало физически достоверным: боковая сила нарастает постепенно, автомобиль плавно реагирует на изменение наклона, а характерные артефакты исчезают. Этот результат подтверждает практическую значимость предложенного механизма и согласуется с физическим представлением о конечной длине релаксации покрышки.

Достоверность реализованной модели покрышки подтверждается также соответствием формы расчётной характеристики теоретической кривой модели Пасейки. Расчётная зависимость боковой силы от угла увода имеет характерный S-образный вид: при малых углах увода сила растёт практически линейно, затем прирост замедляется, и при углах, превышающих предельный, сила выходит на насыщение. Такое поведение совпадает с поведением, предсказываемым формулой (1.1), а также качественно согласуется с экспериментально измеренными характеристиками реальных шин, приводимыми в литературе.

Дополнительным свидетельством корректности служит поведение модели в режиме комбинированного нагружения. При одновременном интенсивном разгоне и повороте суммарная сила, реализуемая покрышкой, ограничивается эллипсом трения, вследствие чего избыточный приводной момент приводит к пробуксовке и снижению боковой силы — наблюдается характерный занос ведущей оси. Аналогично при торможении в повороте чрезмерное тормозное усилие снижает доступную боковую силу. Эти эффекты воспроизводятся моделью без специальных дополнительных условий, как естественное следствие ограничения результирующего вектора силы, что свидетельствует о внутренней согласованности модели.

В качестве критерия валидации модели принималось совпадение формы и пикового значения характеристики боковой силы с эталонной кривой при сохранении расхождения в рабочем диапазоне углов увода в допустимых пределах. Более строгая количественная валидация — сопоставление

расчётных характеристик с экспериментальными данными конкретной шины на едином наборе нагрузок — требует доступа к стендовым измерениям и отнесена к направлениям дальнейшей работы.

4.4 Ограничения и направления дальнейшей работы

Разработанное решение обладает рядом ограничений, обусловленных принятыми упрощениями. Во-первых, пятно контакта покрышки с поверхностью моделируется одной точкой, что не позволяет учитывать неравномерное распределение давления и эффекты, связанные с конечными размерами пятна контакта. Во-вторых, точная идентификация коэффициентов полной формулы Пасейки требует обширной экспериментальной базы, тогда как используемая базовая форма опирается на подобранные эмпирически параметры. В-третьих, модуль не имеет доступа к внутренним оптимизациям конкретного игрового движка, что является обратной стороной его независимости от среды исполнения. В-четвёртых, точное воспроизведение поведения покрышки при различных настройках углов установки колёс — схождения, развала и кастера — затруднено в рамках принятой модели.

Указанные ограничения определяют направления дальнейшего развития. Перспективными представляются: проведение количественных измерений производительности конкурирующих решений на едином стенде для прямого сравнения; переход к многоточечной или щёточной модели контакта для повышения точности; распараллеливание вычислений для эффективной симуляции множества автомобилей, включая трафик под управлением искусственного интеллекта; а также сборка и проверка модуля на реальных целевых платформах — мобильных устройствах и игровых консолях.

Таким образом, в четвёртой главе описана тестовая интеграция модуля в игровой движок Unity, приведены результаты оценки производительности, показавшие высокий запас по бюджету кадра при типичных рабочих частотах, выполнена качественная оценка корректности работы модели и

сформулированы ограничения решения и направления его дальнейшего развития.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы решены все поставленные задачи и достигнута её цель — разработан кросс-платформенный модуль автомобильной симуляции в виде самостоятельной динамически подключаемой библиотеки на языке C++.

Проведён сравнительный анализ основных математических моделей автомобильной покрышки — щёточной модели, модели Пасейки, модели Фиалы и модели Дугоффа. Установлено, что каждая модель имеет свою область применения, определяемую балансом между точностью, вычислительной сложностью и требованиями к исходным данным. На основании анализа в качестве основы модуля обоснованно выбрана модель Пасейки в базовой форме как обеспечивающая наилучшую точность при минимальной вычислительной стоимости и умеренных требованиях к данным.

Описаны и реализованы математические модели всех узлов автомобиля: покрышки с учётом комбинированного нагружения, независимой подвески типа МакФерсон, рулевого управления по принципу Аккермана, открытого и заблокированного дифференциалов, двигателя и коробки передач. Спроектирована модульная архитектура библиотеки и её программный интерфейс на основе стабильного двоичного интерфейса языка C, обеспечивающего совместимость с произвольными вызывающими средами и потокобезопасность. Реализовано численное интегрирование с фиксированным шагом, а для обеспечения устойчивости расчёта угловой скорости колеса применена неявная схема линеаризации реакции покрышки.

Выполнена тестовая интеграция модуля в игровой движок Unity. Тесты подтвердили корректность физической симуляции в широком диапазоне условий. Оценка производительности показала, что при типичной частоте обновления физики затраты на расчёт составляют доли процента бюджета кадра, что свидетельствует о высокой степени оптимизации и наличии значительного запаса для масштабирования.

Основным результатом работы является мультиплатформенный проект библиотеки автомобильной физики, позволяющий без привязки к конкретной архитектуре процессора и конкретному игровому движку выполнять автомобильную симуляцию практически на любой платформе — персональных компьютерах, мобильных устройствах и игровых консолях. Для сборки библиотеки под целевую платформу достаточно соответствующего набора инструментов разработки. Существенным научным вкладом является введение динамического коэффициента скольжения, обеспечивающего симуляцию динамического перехода боковой силы трения покрышки и учитывающего конечное время достижения предела сцепления, что устраняет нефизичное скатывание автомобиля с наклонных поверхностей. Модульная организация системы обеспечивает её лёгкую расширяемость и быстрое исправление ошибок, а пример интеграции в Unity подтверждает простоту встраивания библиотеки в готовую среду.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Pacejka, H. B. Tire and Vehicle Dynamics / H. B. Pacejka. — 3rd ed. — Oxford : Butterworth-Heinemann, 2012. — 672 p.
2. Gillespie, T. D. Fundamentals of Vehicle Dynamics / T. D. Gillespie. — Warrendale : SAE International, 1992. — 519 p.
3. Jazar, R. N. Vehicle Dynamics: Theory and Application / R. N. Jazar. — 3rd ed. — Cham : Springer, 2017. — 1068 p.
4. Rajamani, R. Vehicle Dynamics and Control / R. Rajamani. — 2nd ed. — New York : Springer, 2012. — 498 p.
5. Kiencke, U. Automotive Control Systems: For Engine, Driveline and Vehicle / U. Kiencke, L. Nielsen. — 2nd ed. — Berlin : Springer, 2005. — 512 p.
6. Milliken, W. F. Race Car Vehicle Dynamics / W. F. Milliken, D. L. Milliken. — Warrendale : SAE International, 1995. — 890 p.
7. Fiala, E. Seitenkräfte am rollenden Luftreifen / E. Fiala // VDI-Zeitschrift. — 1954. — S. 973–979.
8. Dugoff, H. An analysis of tire traction properties and their influence on vehicle dynamic performance / H. Dugoff, P. Fancher, L. Segel // SAE Technical Paper 700377. — 1970.
9. Genta, G. Motor Vehicle Dynamics: Modeling and Simulation / G. Genta. — Singapore : World Scientific, 1997. — 539 p.
10. Wong, J. Y. Theory of Ground Vehicles / J. Y. Wong. — 4th ed. — Hoboken : John Wiley & Sons, 2008. — 592 p.
11. Blundell, M. The Multibody Systems Approach to Vehicle Dynamics / M. Blundell, D. Harty. — 2nd ed. — Oxford : Butterworth-Heinemann, 2015. — 768 p.
12. Tasora, A. Chrono: An Open Source Multi-physics Dynamics Engine / A. Tasora [et al.] // Lecture Notes in Computer Science. — 2016. — Vol. 9611. — P. 19–49.

ПРИЛОЖЕНИЕ А

(обязательное)

Программный интерфейс библиотеки на основе C-ABI

Листинг А.1 — car_physics.h

```
/*
 * car_physics.h
 *
 * Public C ABI for the universal car-simulation module.
 *
 * Design: data-in / data-out. The DLL contains ONLY the math (engine,
 * gearbox, clutch, differential, brakes, steering, suspension, slip, tire
 * and wheel acceleration). It never touches a game engine: it does no
 * raycasting, applies no forces and reads no transforms by itself.
 *
 * The HOST (Unity, Unreal, a Python lab harness, ...) is responsible for:
 *   - performing the wheel ground raycasts,
 *   - reading the rigid-body point velocity at each contact,
 *   - feeding the world-space basis of every wheel root,
 *   - applying the forces / torques the DLL returns.
 *
 * Per simulation tick the host calls, in order:
 *   1) carsim_update_drivetrain(...) -> steer angles + drivetrain telemetry
 *   2) host applies steer angles to the wheel roots, raycasts, gathers
 *      contact data and point velocities
 *   3) carsim_update_wheels(...) -> forces to apply + visuals + telemetry
 *   4) host applies the returned forces at the returned points
 *
 * All vectors are expressed in the host's own coordinate system. The module
 * is coordinate-system agnostic because the host supplies the wheel-root
 * basis vectors (right / up / forward) directly instead of a quaternion.
 *
 * The ABI is plain C so it can be consumed from C# (P/Invoke), Unreal C++,
 * Python (ctypes), etc.
 */
#ifndef CAR_PHYSICS_H
#define CAR_PHYSICS_H

#ifdef _WIN32
#ifdef CARPHYSICS_BUILD_DLL
#define CARPHYSICS_API __declspec(dllexport)
#else
#define CARPHYSICS_API __declspec(dllimport)
#endif
#define CARPHYSICS_CALL __cdecl
#else
#define CARPHYSICS_API __attribute__((visibility("default")))
#define CARPHYSICS_CALL
#endif

#ifdef __cplusplus
extern "C" {
#endif

/* The drivetrain math (differential / brakes / steering) is wired for a
 * 4-wheel car. Wheel index convention, matching the original C# project:
 *   0 = front-left, 1 = front-right, 2 = rear-left, 3 = rear-right. */
#define CARSIM_WHEEL_COUNT 4

typedef struct CP_Vec3 { float x, y, z; } CP_Vec3;
```

```

/* Piecewise-linear curve. Keyframes must be sorted by time ascending.
 * Evaluation clamps to the first/last value outside the key range. The host
 * owns the arrays only for the duration of carsim_create; the module copies
 * them internally. */
typedef struct CP_Curve {
    const float* times;
    const float* values;
    int          count;
} CP_Curve;

typedef struct CP_EngineInfo {
    CP_Curve torqueCurve;      /* torque vs rpm */
    CP_Vec3  engineOrientation; /* axis used for the neutral-gear body torque */
    float    idleRpm;
    float    maxRpm;
    float    mul;              /* engineMul */
    float    frictionCoeff;    /* engineFrictionCoefficient */
    float    startFriction;
    float    inertia;          /* engineInertia */
} CP_EngineInfo;

typedef struct CP_GearboxInfo {
    const float* ratios;      /* index 0 is reverse/neutral as authored */
    int          gearCount;
    float        shiftTime;   /* seconds spent in neutral while shifting */
} CP_GearboxInfo;

typedef struct CP_ClutchInfo {
    float stiffness;
    float capacity;
    float damping;
} CP_ClutchInfo;

typedef struct CP_DifferentialInfo {
    int    isLocked;      /* 0 = open, non-zero = locked */
    float  ratio;         /* differentialRatio */
} CP_DifferentialInfo;

typedef struct CP_BrakesInfo {
    CP_Curve brakeTorqueCurve; /* multiplier vs wheel angular speed */
    float    maxTorque;
    float    biasFront;       /* brakeBias[0] */
    float    biasRear;        /* brakeBias[1] */
} CP_BrakesInfo;

typedef struct CP_SteeringInfo {
    float turnRadius;
    float steerForce;
    float maxCorrectionAngle;
    float correctionSpeed;
} CP_SteeringInfo;

typedef struct CP_WheelInfo {
    float restLength;
    float suspensionStiffness;
    float damperStiffness;
    float slipAnglePeak;      /* slip angle (deg) at peak lateral force; sets
Pacejka B_lat */
    float camber;             /* exposed for visuals; not used by the math */
    float caster;             /* idem */
    float longitudinalCoeff;  /* longitudinal peak-friction factor: Dx = coeff * Fz
*/
    float lateralCoeff;       /* lateral peak-friction factor:      Dy = coeff * Fz
*/
    float wheelRadius;
    float wheelMass;          /* wheelInertia = wheelRadius^2 * wheelMass */
    float longFrictionCoeff;

```

```

float relaxationLength;

/* ---- Pacejka Magic Formula shape parameters ----
 * Per axis force:  $F(x) = D \sin(C \operatorname{atan}(Bx - E(Bx - \operatorname{atan}(Bx))))$ .
 * D comes from the friction factor * normal load (above). The stiffness
 * factor B is derived so the curve peaks at the configured peak slip
 * (longSlipPeak for the slip ratio, slipAnglePeak for the slip angle):
 *  $B = \tan(\pi / (2C)) / \text{peakSlip}$ . */
float longSlipPeak; /* slip ratio at peak longitudinal force (e.g. 0.12)
*/
float pacejkaShapeLong; /* C_x (e.g. 1.65) */
float pacejkaCurveLong; /* E_x (e.g. 0.96) */
float pacejkaShapeLat; /* C_y (e.g. 1.35) */
float pacejkaCurveLat; /* E_y (e.g. 0.96) */
} CP_WheelInfo;

typedef struct CP_CarConfig {
    CP_EngineInfo engine;
    CP_GearboxInfo gearbox;
    CP_ClutchInfo clutch;
    CP_DifferentialInfo differential;
    CP_BrakesInfo brakes;
    CP_SteeringInfo steering;
    CP_WheelInfo wheels[CARSIM_WHEEL_COUNT];

    /* Ackermann geometry, measured once by the host from the wheel-root
     * world positions:
     * wheelBase = distance(wheel[0], wheel[2])
     * rearTrack = distance(wheel[2], wheel[3]) */
    float wheelBase;
    float rearTrack;
} CP_CarConfig;

/* ---- per-tick drivetrain phase ---- */

typedef struct CP_DrivetrainInput {
    float dt; /* fixed timestep, seconds */
    float throttle; /* 0..1 */
    float brake; /* 0..1 */
    float steer; /* -1..1 */
    int gearUp; /* 1 on the frame the shift-up was requested, else 0 */
    int gearDown; /* 1 on the frame the shift-down was requested, else 0 */
} CP_DrivetrainInput;

typedef struct CP_DrivetrainOutput {
    float steerAngles[CARSIM_WHEEL_COUNT]; /* degrees, host applies to roots */
    CP_Vec3 neutralBodyTorque; /* apply when applyNeutralTorque */
    int applyNeutralTorque;
    float engineRpm;
    float engineAngularVelocity;
    int currentGear;
    float clutchTorque;
    float clutchLock;
} CP_DrivetrainOutput;

/* ---- per-tick wheel phase ---- */

typedef struct CP_WheelState {
    CP_Vec3 position; /* wheel-root world position */
    CP_Vec3 right; /* wheel-root world basis (after steering) */
    CP_Vec3 up;
    CP_Vec3 forward;
    int hit; /* 1 if the ground raycast hit */
    CP_Vec3 hitPoint; /* world contact point */
    CP_Vec3 hitNormal; /* world contact normal */
    CP_Vec3 pointVelocity; /* rigid-body velocity at hitPoint, world space */

```

```

} CP_WheelState;

typedef struct CP_WheelInput {
    float      dt;
    CP_WheelState wheels[CARSIM_WHEEL_COUNT];
} CP_WheelInput;

typedef struct CP_WheelOutput {
    /* Forces to apply with the engine equivalent of AddForceAtPosition.
     * applyForce already combines suspension + tire force for the wheel. */
    CP_Vec3 applyForce[CARSIM_WHEEL_COUNT];
    CP_Vec3 applyPoint[CARSIM_WHEEL_COUNT];

    /* Visuals, ready to apply on the host transforms. */
    CP_Vec3 visualPosition[CARSIM_WHEEL_COUNT];
    float   spinEulerX[CARSIM_WHEEL_COUNT]; /* rotating part local euler X */
    float   steerEulerY[CARSIM_WHEEL_COUNT]; /* visual local euler Y */

    /* Telemetry. */
    float angularVelocity[CARSIM_WHEEL_COUNT];
    float suspensionForce[CARSIM_WHEEL_COUNT];
    float currentLength[CARSIM_WHEEL_COUNT];
    CP_Vec3 linearVelocity[CARSIM_WHEEL_COUNT]; /* wheel-local, for telemetry */
    float slipAngle[CARSIM_WHEEL_COUNT];
    float lateralAcceleration[CARSIM_WHEEL_COUNT];
    float slipForceLong[CARSIM_WHEEL_COUNT];
    float slipForceLat[CARSIM_WHEEL_COUNT];
    float normalizedTireMagnitude[CARSIM_WHEEL_COUNT]; /* drives skid sound */
    float fx[CARSIM_WHEEL_COUNT];
} CP_WheelOutput;

/* Opaque simulation handle. */
typedef void* CP_Handle;

/* Create a simulation from a config. Returns NULL on invalid input.
 * The config (including curve and gear-ratio arrays) is deep-copied. */
CARPHYSICS_API CP_Handle CARPHYSICS_CALL carsim_create(const CP_CarConfig* config);

/* Destroy a simulation created by carsim_create. NULL is ignored. */
CARPHYSICS_API void CARPHYSICS_CALL carsim_destroy(CP_Handle handle);

/* Phase 1: steering + engine + gearbox + clutch + differential + brakes. */
CARPHYSICS_API void CARPHYSICS_CALL carsim_update_drivetrain(
    CP_Handle handle, const CP_DrivetrainInput* in, CP_DrivetrainOutput* out);

/* Phase 2: suspension + wheel acceleration + slip + tire + visuals. */
CARPHYSICS_API void CARPHYSICS_CALL carsim_update_wheels(
    CP_Handle handle, const CP_WheelInput* in, CP_WheelOutput* out);

/* Version string, e.g. "1.0.0". */
CARPHYSICS_API const char* CARPHYSICS_CALL carsim_version(void);

#ifdef __cplusplus
} /* extern "C" */
#endif

#endif /* CAR_PHYSICS_H */

```

ПРИЛОЖЕНИЕ Б

(обязательное)

Реализация математических моделей узлов автомобиля

Листинг Б.1 — models.h

```
/*
 * models.h - C++ port of the C# physics models / wheel components.
 *
 * Each class reproduces the math of its Unity counterpart 1:1. All engine
 * coupling (raycasts, AddForce, GetPointVelocity, Transform reads) is
 * removed: inputs arrive as plain values, outputs are returned as plain
 * values, and the host applies them.
 */
#ifndef CARSIM_MODELS_H
#define CARSIM_MODELS_H

#include "math_util.h"
#include "car_physics.h"

namespace carsim {

/* ----- engine */
class EngineModel {
public:
    void init(const CP_EngineInfo& info);

    /* Returns true and fills outBodyTorque when the gear is neutral (0). */
    bool update(float throttle, float clutchTorque, int currentGear, float dt,
               Vec3& outBodyTorque);

    float angularVelocity() const { return m_angularVelocity; }
    float rpm() const { return m_rpm; }
    float maxEngineTorque() const { return m_torqueCurve.maxValue(); }

private:
    CP_EngineInfo m_info{};
    Curve m_torqueCurve;
    float m_rpm = 0.0f;
    float m_angularVelocity = 100.0f;
};

/* ----- gearbox */
class GearboxModel {
public:
    void init(const CP_GearboxInfo& info);

    void requestShiftUp();
    void requestShiftDown();
    void tick(float dt); /* advances shift timer */

    float currentGearRatio() const { return m_ratios[m_currentGear]; }
    int currentGear() const { return m_currentGear; }

    float inputShaftVelocity(float diffShaftVelocity) const {
        return diffShaftVelocity * currentGearRatio();
    }
    float outputTorque(float inputTorque) const {
        return inputTorque * m_ratios[m_currentGear];
    }
};

}
```

```

    }

private:
    std::vector<float> m_ratios;
    float m_shiftTime = 0.0f;
    int m_currentGear = 1;
    int m_targetGear = 1;
    bool m_isShifting = false;
    float m_shiftTimer = 0.0f;
};

/* ----- clutch */
class ClutchModel {
public:
    void init(const CP_ClutchInfo& info, float maxEngineTorque);

    void update(float engineAngularVelocity, float currentGearRatio,
                float gearBoxInputShaftVelocity);

    float torque() const { return m_clutchTorque; }
    float lock() const { return m_clutchLock; }

private:
    CP_ClutchInfo m_info{};
    float m_maxEngineTorque = 0.0f;
    float m_clutchTorque = 0.0f;
    float m_clutchLock = 0.0f;
};

/* ----- differential */
class DifferentialModel {
public:
    void init(const CP_DifferentialInfo& info, const CP_WheelInfo& wheel0);

    void update(float inputTorque, const float angularVelocities[CARSIM_WHEEL_COUNT],
                float dt);

    float inputShaftVelocity() const {
        return (m_angularVelocityL + m_angularVelocityR) * 0.5f * m_info.ratio;
    }
    const float* outputTorque() const { return m_outputTorque; }

private:
    CP_DifferentialInfo m_info{};
    float m_wheelInertia = 0.0f;
    float m_outputTorque[CARSIM_WHEEL_COUNT] = { 0 };
    float m_angularVelocityL = 0.0f;
    float m_angularVelocityR = 0.0f;
};

/* ----- brakes */
class BrakesModel {
public:
    void init(const CP_BrakesInfo& info);

    void update(float brakeInput, const float angularVelocities[CARSIM_WHEEL_COUNT]);

    const float* brakeTorque() const { return m_brakeTorque; }

private:
    CP_BrakesInfo m_info{};
    Curve m_curve;
    float m_brakeTorque[CARSIM_WHEEL_COUNT] = { 0 };
};

```

```

};

/* ----- steering */
class SteeringModel {
public:
    void init(const CP_SteeringInfo& info, float wheelBase, float rearTrack);

    void update(float inputSteering,
                const float lateralAccelerations[CARSIM_WHEEL_COUNT],
                float dt);

    const float* steerAngles() const { return m_steerAngles; }

private:
    CP_SteeringInfo m_info{};
    float m_wheelBase = 0.0f;
    float m_rearTrack = 0.0f;
    float m_ackermannAngleL = 0.0f;
    float m_ackermannAngleR = 0.0f;
    float m_steerAngles[CARSIM_WHEEL_COUNT] = { 0 };
};

/* ----- per-wheel components --- */

class SuspensionWheel {
public:
    void init(const CP_WheelInfo& info) { m_info = info; }

    /* Computes suspension force + local point velocity, returns the world
     * force to add at the contact point (= suspensionForce * up). */
    Vec3 update(const CP_WheelState& w, float dt);

    float suspensionForce() const { return m_suspensionForce; }
    float currentLength() const { return m_currentLength; }
    const Vec3& linearVelocity() const { return m_linearVelocity; }

private:
    CP_WheelInfo m_info{};
    float m_suspensionForce = 0.0f;
    float m_currentLength = 0.0f;
    float m_lastLength = 0.0f;
    Vec3 m_linearVelocity;
};

class AccelerationWheel {
public:
    void init(const CP_WheelInfo& info) {
        m_info = info;
        m_wheelInertia = info.wheelRadius * info.wheelRadius * info.wheelMass;
        m_bLong = magicStiffnessFromPeak(info.pacejkaShapeLong, info.longSlipPeak);
    }

    /* Integrates wheel spin. The longitudinal tire reaction is linearised about
     * the current wheel speed (implicit) for stability, which needs the normal
     * load via suspensionForce and the derived Pacejka stiffness m_bLong. */
    void update(float fx, float driveTorque, float brakeTorque,
                float groundForwardSpeed, float suspensionForce, float dt);

    float angularVelocity() const { return m_angularVelocity; }

private:
    CP_WheelInfo m_info{};
    float m_wheelInertia = 0.0f;

```

```

    float m_bLong = 0.0f;    /* derived Pacejka longitudinal stiffness factor */
    float m_angularVelocity = 0.0f;
};

class SlipWheel {
public:
    void init(const CP_WheelInfo& info) {
        m_info = info;
        m_wheelInertia = info.wheelRadius * info.wheelRadius * info.wheelMass;
    }

    void update(const Vec3& linearVelocity, float suspensionForce,
               float angularVelocity, float dt);

    float slipRatio() const      { return m_slipRatio; }           /* longitudinal,
dimensionless */
    float slipAngleRad() const   { return m_dynamicSlipAngle; }    /* lateral, radians
(relaxed) */
    float slipAngle() const      { return m_dynamicSlipAngle * RAD2DEG; } /* telemetry,
degrees */
    float lateralAcceleration() const { return m_lateralAcceleration; }

private:
    CP_WheelInfo m_info{};
    float m_wheelInertia = 0.0f;
    float m_slipRatio = 0.0f;
    float m_dynamicSlipAngle = 0.0f; /* relaxed slip angle, radians */
    float m_lateralAcceleration = 0.0f;
};

class TireWheel {
public:
    void init(const CP_WheelInfo& info);

    /* Pacejka Magic Formula tire. Takes the longitudinal slip ratio and the
    * (relaxed) slip angle in radians; returns the world tire force to add at
    * the contact point. */
    Vec3 update(const CP_WheelState& w, float slipRatio, float slipAngleRad,
               float suspensionForce);

    float fx() const { return m_fx; }
    float normalizedMagnitude() const { return m_normalizedMagnitude; }

private:
    CP_WheelInfo m_info{};
    float m_bLong = 0.0f; /* derived Pacejka stiffness factors */
    float m_bLat = 0.0f;
    float m_fx = 0.0f;
    float m_normalizedMagnitude = 0.0f;
};

class VisualWheel {
public:
    void init() {}

    /* Accumulates spin and produces euler angles ready for the host. */
    void update(float angularVelocity, float steerAngle, bool isOppositeSide,
               float dt, float& outSpinEulerX, float& outSteerEulerY);

private:
    float m_currentAngle = 0.0f;
};

} // namespace carsim

```



```
#endif // CARSIM_MODELS_H
```

Листинг Б.2 — models.cpp

```
/*
 * models.cpp - implementations ported 1:1 from the C# physics models.
 */
#include "models.h"

#include <algorithm>
#include <limits>

namespace carsim {

/* ----- engine */
void EngineModel::init(const CP_EngineInfo& info) {
    m_info = info;
    m_torqueCurve.assign(info.torqueCurve);
    m_rpm = 0.0f;
    m_angularVelocity = 100.0f; // matches the original constructor
}

bool EngineModel::update(float throttle, float clutchTorque, int currentGear,
                        float dt, Vec3& outBodyTorque) {
    float maxEffectiveTorque = m_torqueCurve.evaluate(m_rpm) * m_info.mul;
    float engineFriction = (m_rpm * m_info.frictionCoeff) + m_info.startFriction;
    float engineTorque = maxEffectiveTorque * throttle - engineFriction;
    float engineAcceleration = (engineTorque - clutchTorque) / m_info.inertia;

    m_angularVelocity += engineAcceleration * dt;
    m_rpm = m_angularVelocity * RAD_TO_RPM; // computed before clamp, as in C#
    m_angularVelocity = clampf(m_angularVelocity,
                              m_info.idleRpm * RPM_TO_RAD,
                              m_info.maxRpm * RPM_TO_RAD);

    if (currentGear == 0) {
        outBodyTorque = Vec3(m_info.engineOrientation) * (engineTorque * 2.0f);
        return true;
    }
    return false;
}

/* ----- gearbox */
void GearboxModel::init(const CP_GearboxInfo& info) {
    m_ratios.clear();
    if (info.ratios && info.gearCount > 0) {
        m_ratios.assign(info.ratios, info.ratios + info.gearCount);
    }
    m_shiftTime = info.shiftTime;
    m_currentGear = 1;
    m_targetGear = 1;
    m_isShifting = false;
    m_shiftTimer = 0.0f;
}

void GearboxModel::requestShiftUp() {
    if (!m_isShifting && m_currentGear < static_cast<int>(m_ratios.size()) - 1) {
        m_targetGear = m_currentGear + 1;
        m_isShifting = true;
        m_shiftTimer = m_shiftTime;
        m_currentGear = 1; // neutral while shifting, as in the original
    }
}
```

```

void GearboxModel::requestShiftDown() {
    if (!m_isShifting && m_currentGear != 0) {
        m_targetGear = m_currentGear - 1;
        m_isShifting = true;
        m_shiftTimer = m_shiftTime;
        m_currentGear = 1;
    }
}

void GearboxModel::tick(float dt) {
    if (!m_isShifting) return;
    m_shiftTimer -= dt;
    if (m_shiftTimer <= 0.0f) {
        m_currentGear = m_targetGear;
        m_isShifting = false;
    }
}

/* ----- clutch */
void ClutchModel::init(const CP_ClutchInfo& info, float maxEngineTorque) {
    m_info = info;
    m_maxEngineTorque = maxEngineTorque;
    m_clutchTorque = 0.0f;
    m_clutchLock = 0.0f;
}

void ClutchModel::update(float engineAngularVelocity, float currentGearRatio,
                        float gearBoxInputShaftVelocity) {
    float clutchMaxTorque = m_maxEngineTorque * m_info.capacity;
    float clutchSlip = (engineAngularVelocity - gearBoxInputShaftVelocity)
        * std::fabs(signf(currentGearRatio));
    m_clutchLock = currentGearRatio == 0.0f
        ? 0.0f
        : mapRangeClamped(engineAngularVelocity * RAD_TO_RPM,
                        1000.0f, 1300.0f, 0.0f, 1.0f);
    float clt = clampf(clutchSlip * m_clutchLock * m_info.stiffness,
        -clutchMaxTorque, clutchMaxTorque);
    m_clutchTorque = clt + ((m_clutchTorque - clt) * m_info.damping);
}

/* ----- differential */
void DifferentialModel::init(const CP_DifferentialInfo& info,
                        const CP_WheelInfo& wheel0) {
    m_info = info;
    m_wheelInertia = wheel0.wheelRadius * wheel0.wheelRadius * wheel0.wheelMass;
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) m_outputTorque[i] = 0.0f;
}

void DifferentialModel::update(float inputTorque,
                        const float angularVelocities[CARSIM_WHEEL_COUNT],
                        float dt) {
    m_angularVelocityL = angularVelocities[2];
    m_angularVelocityR = angularVelocities[3];

    if (!m_info.isLocked) {
        float vel = (m_angularVelocityL - m_angularVelocityR) * 0.5f / dt *
m_wheelInertia;
        float base = inputTorque * 0.5f * m_info.ratio;
        m_outputTorque[0] = base - vel;
        m_outputTorque[1] = base + vel;
        m_outputTorque[2] = base - vel;
        m_outputTorque[3] = base + vel;
    } else {
        float base = inputTorque * m_info.ratio * 0.5f;
        for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) m_outputTorque[i] = base;
    }
}

```

```

}

/* ----- brakes */
void BrakesModel::init(const CP_BrakesInfo& info) {
    m_info = info;
    m_curve.assign(info.brakeTorqueCurve);
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) m_brakeTorque[i] = 0.0f;
}

void BrakesModel::update(float brakeInput,
                        const float angularVelocities[CARSIM_WHEEL_COUNT]) {
    // Index convention: 0=FL, 1=FR, 2=RL, 3=RR. The brake-torque curve is
    // looked up per axle from that axle's mean wheel speed.
    float frontSpeed = std::fabs((angularVelocities[0] + angularVelocities[1]) *
0.5f);
    float rearSpeed = std::fabs((angularVelocities[2] + angularVelocities[3]) *
0.5f);

    m_brakeTorque[0] = brakeInput * m_info.biasFront * m_info.maxTorque *
m_curve.evaluate(frontSpeed);
    m_brakeTorque[1] = m_brakeTorque[0];
    m_brakeTorque[2] = brakeInput * m_info.biasRear * m_info.maxTorque *
m_curve.evaluate(rearSpeed);
    m_brakeTorque[3] = m_brakeTorque[2];
}

/* ----- steering */
void SteeringModel::init(const CP_SteeringInfo& info, float wheelBase,
                        float rearTrack) {
    m_info = info;
    m_wheelBase = wheelBase;
    m_rearTrack = rearTrack;
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) m_steerAngles[i] = 0.0f;
}

void SteeringModel::update(float inputSteering,
                        const float lateralAccelerations[CARSIM_WHEEL_COUNT],
                        float dt) {
    float in = inputSteering;

    // Ackermann
    float denomL = in > 0 ? m_info.turnRadius + m_rearTrack / 2.0f
                        : m_info.turnRadius - m_rearTrack / 2.0f;
    float denomR = in > 0 ? m_info.turnRadius - m_rearTrack / 2.0f
                        : m_info.turnRadius + m_rearTrack / 2.0f;
    m_ackermannAngleL = RAD2DEG * std::atan(m_wheelBase / denomL) * in *
m_info.steerForce;
    m_ackermannAngleR = RAD2DEG * std::atan(m_wheelBase / denomR) * in *
m_info.steerForce;

    // Correction from lateral acceleration
    float correctionFactor = m_info.correctionSpeed * dt;
    m_ackermannAngleL -= lateralAccelerations[1] * correctionFactor;
    m_ackermannAngleR -= lateralAccelerations[0] * correctionFactor;
    m_ackermannAngleL = clampf(m_ackermannAngleL, -m_info.maxCorrectionAngle,
m_info.maxCorrectionAngle);
    m_ackermannAngleR = clampf(m_ackermannAngleR, -m_info.maxCorrectionAngle,
m_info.maxCorrectionAngle);

    float t = m_info.correctionSpeed * dt;
    m_steerAngles[0] = lerp(m_steerAngles[0], m_ackermannAngleR, t);
    m_steerAngles[1] = lerp(m_steerAngles[1], m_ackermannAngleL, t);
    // rear wheels (2,3) stay 0
}

```

```

/* ----- suspension */
Vec3 SuspensionWheel::update(const CP_WheelState& w, float dt) {
    Vec3 up(w.up);
    Vec3 pos(w.position);
    Vec3 hitPoint(w.hitPoint);

    m_currentLength = magnitude(pos - (hitPoint + up * m_info.wheelRadius));

    float springForce = (m_info.restLength - m_currentLength) *
m_info.suspensionStiffness;
    float damperForce = (m_lastLength - m_currentLength) / dt *
m_info.damperStiffness;
    m_suspensionForce = std::max(0.0f, springForce + damperForce);
    m_lastLength = m_currentLength;

    m_linearVelocity = inverseTransformDirection(Vec3(w.pointVelocity),
                                                Vec3(w.right), up, Vec3(w.forward));

    return up * m_suspensionForce; // force to add at hitPoint
}

/* ----- acceleration */
void AccelerationWheel::update(float fx, float driveTorque, float brakeTorque,
                                float groundForwardSpeed, float suspensionForce,
                                float dt) {
    // The longitudinal tire force is very stiff w.r.t. wheel speed (its slope
    // is B*C*D, scaled up at low ground speed). Integrating it explicitly makes
    // the wheel speed oscillate and blow up at a fixed timestep. We instead
    // treat that dependence implicitly: linearise the tire reaction around the
    // current wheel speed and solve for the new speed. This is unconditionally
    // stable and lets the wheel settle on its rolling speed instead of spinning
    // up to the clamp.
    const float kVelFloor = 1.0f; // m/s, keeps the slope finite near standstill
    float denom = std::max(std::fabs(groundForwardSpeed), kVelFloor);

    float reactionTorque = fx * m_info.wheelRadius; // last tick's tire
reaction

    float slipRatio = (m_angularVelocity * m_info.wheelRadius - groundForwardSpeed) /
denom;
    float dLong = m_info.longitudinalCoeff * std::max(0.0f, suspensionForce);
    float forceSlope = magicFormulaSlope(slipRatio, m_bLong, m_info.pacejkaShapeLong,
                                         dLong, m_info.pacejkaCurveLong);
    // dTorque/dOmega = R * dF/dslip * dslip/dOmega, dslip/dOmega = R/denom.
    float torqueSlope = std::max(0.0f, forceSlope) * m_info.wheelRadius *
m_info.wheelRadius / denom;

    float deltaOmega = (driveTorque - reactionTorque) / (m_wheelInertia / dt +
torqueSlope);
    m_angularVelocity += deltaOmega;

    // Braking always opposes the current spin and can at most bring the wheel
    // to a stop this tick (the min() prevents braking from reversing it).
    float brakeDelta = brakeTorque / m_wheelInertia * dt; // >= 0
    m_angularVelocity -= signf(m_angularVelocity) *
std::min(std::fabs(m_angularVelocity), brakeDelta);

    m_angularVelocity = clampf(m_angularVelocity, -360.0f, 360.0f);
}

/* ----- slip */
void SlipWheel::update(const Vec3& linearVelocity, float suspensionForce,
                        float angularVelocity, float dt) {
    // A velocity floor keeps the slip ratio / slip angle finite near standstill

```

```

// (both are otherwise singular as the forward speed approaches zero).
const float kVelFloor = 1.0f; // m/s
float vLong = linearVelocity.z;
float vLat = linearVelocity.x;
float denom = std::max(std::fabs(vLong), kVelFloor);

// longitudinal slip ratio: (wheel surface speed - ground speed) / ground speed
float wheelSurfaceSpeed = angularVelocity * m_info.wheelRadius;
m_slipRatio = (wheelSurfaceSpeed - vLong) / denom;
m_slipRatio = clampf(m_slipRatio, -4.0f, 4.0f);

// lateral slip angle (radians), relaxed over the relaxation length so the
// tire force builds up over travelled distance rather than instantly.
float slipAngle = std::atan(-vLat / denom);
float relax = m_info.relaxationLength > 1e-4f
    ? clamp01(std::fabs(vLong) * dt / m_info.relaxationLength)
    : 1.0f;
m_dynamicSlipAngle += (slipAngle - m_dynamicSlipAngle) * relax;
m_dynamicSlipAngle = clampf(m_dynamicSlipAngle, -PI * 0.5f, PI * 0.5f);

float mag = magnitude(linearVelocity);
m_lateralAcceleration = (mag * mag / m_info.wheelRadius) *
std::tan(m_dynamicSlipAngle);
}

/* ----- tire */
void TireWheel::init(const CP_WheelInfo& info) {
    m_info = info;
    m_bLong = magicStiffnessFromPeak(info.pacejkaShapeLong, info.longSlipPeak);
    m_bLat = magicStiffnessFromPeak(info.pacejkaShapeLat, info.slipAnglePeak *
DEG2RAD);
    m_fx = 0.0f;
    m_normalizedMagnitude = 0.0f;
}

Vec3 TireWheel::update(const CP_WheelState& w, float slipRatio, float slipAngleRad,
    float suspensionForce) {
    Vec3 forwardProj = normalized(projectOnPlane(Vec3(w.forward), Vec3(w.hitNormal)));
    Vec3 sideProj = normalized(projectOnPlane(Vec3(w.right), Vec3(w.hitNormal)));

    // Peak forces scale with normal load (friction-factor * Fz).
    float dLong = m_info.longitudinalCoeff * suspensionForce;
    float dLat = m_info.lateralCoeff * suspensionForce;

    float fx0 = magicFormula(slipRatio, m_bLong, m_info.pacejkaShapeLong, dLong,
m_info.pacejkaCurveLong);
    float fy0 = magicFormula(slipAngleRad, m_bLat, m_info.pacejkaShapeLat, dLat,
m_info.pacejkaCurveLat);

    // Combined slip: keep the resultant inside the friction ellipse so the tire
    // cannot deliver more than its limit when slipping in both directions.
    float gx = dLong > 1e-4f ? fx0 / dLong : 0.0f;
    float gy = dLat > 1e-4f ? fy0 / dLat : 0.0f;
    float g = std::sqrt(gx * gx + gy * gy);
    float scale = g > 1.0f ? 1.0f / g : 1.0f;

    m_fx = fx0 * scale;
    float fY = fy0 * scale;

    // Skid intensity: how close the tire is to (or beyond) its friction limit.
    m_normalizedMagnitude = clamp01(g);

    return forwardProj * m_fx + sideProj * fY; // force to add at hitPoint
}

```

```

/* ----- visual */
void VisualWheel::update(float angularVelocity, float steerAngle,
                        bool isOppositeSide, float dt, float& outSpinEulerX,
                        float& outSteerEulerY) {
    m_currentAngle += angularVelocity * RAD2DEG * dt;
    m_currentAngle = std::fmod(m_currentAngle, 360.0f);

    outSpinEulerX = isOppositeSide ? -m_currentAngle : m_currentAngle;
    outSteerEulerY = isOppositeSide ? steerAngle + 180.0f : steerAngle;
}

} // namespace carsim

```

ПРИЛОЖЕНИЕ В

(обязательное)

Ядро симуляции и точка входа динамической библиотеки

Листинг В.1 — math_util.h

```
/*
 * math_util.h - engine-independent vector math and helpers.
 *
 * Mirrors the small subset of UnityEngine.Mathf / Vector3 used by the
 * original C# models so the ported formulas stay identical.
 */
#ifndef CARSIM_MATH_UTIL_H
#define CARSIM_MATH_UTIL_H

#include <cmath>
#include <vector>
#include "car_physics.h"

namespace carsim {

constexpr float PI = 3.14159265358979323846f;
constexpr float RPM_TO_RAD = PI * 2.0f / 60.0f;
constexpr float RAD_TO_RPM = 1.0f / RPM_TO_RAD;
constexpr float DEG2RAD = PI / 180.0f;
constexpr float RAD2DEG = 180.0f / PI;

struct Vec3 {
    float x = 0.f, y = 0.f, z = 0.f;

    Vec3() = default;
    Vec3(float x_, float y_, float z_) : x(x_), y(y_), z(z_) {}
    Vec3(const CP_Vec3& v) : x(v.x), y(v.y), z(v.z) {}

    CP_Vec3 toC() const { return CP_Vec3{ x, y, z }; }
};

inline Vec3 operator+(const Vec3& a, const Vec3& b) { return { a.x + b.x, a.y + b.y, a.z + b.z }; }
inline Vec3 operator-(const Vec3& a, const Vec3& b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
inline Vec3 operator*(const Vec3& a, float s) { return { a.x * s, a.y * s, a.z * s }; }
inline Vec3 operator*(float s, const Vec3& a) { return a * s; }

inline float dot(const Vec3& a, const Vec3& b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
inline float magnitude(const Vec3& a) { return std::sqrt(dot(a, a)); }

inline Vec3 normalized(const Vec3& a) {
    float m = magnitude(a);
    return m > 1e-8f ? a * (1.0f / m) : Vec3{ 0.f, 0.f, 0.f };
}

/* Unity's Vector3.ProjectOnPlane(v, n) = v - n * (dot(v,n)/dot(n,n)). */
inline Vec3 projectOnPlane(const Vec3& v, const Vec3& n) {
    float nn = dot(n, n);
    if (nn < 1e-12f) return v;
    return v - n * (dot(v, n) / nn);
}
```

```

}

/* Unity's Transform.InverseTransformDirection for an orthonormal basis:
 * express world direction v in the local (right, up, forward) frame. */
inline Vec3 inverseTransformDirection(const Vec3& v, const Vec3& right,
                                     const Vec3& up, const Vec3& forward) {
    return { dot(v, right), dot(v, up), dot(v, forward) };
}

/* Unity Mathf helpers. */
inline float clampf(float v, float lo, float hi) {
    return v < lo ? lo : (v > hi ? hi : v);
}

inline float clamp01(float v) { return clampf(v, 0.0f, 1.0f); }

inline float lerp(float a, float b, float t) { return a + (b - a) * clamp01(t); }

inline float inverseLerp(float a, float b, float v) {
    if (a == b) return 0.0f;
    return clamp01((v - a) / (b - a));
}

/* Mathf.Sign: returns +1 for v >= 0, -1 otherwise. */
inline float signif(float v) { return v >= 0.0f ? 1.0f : -1.0f; }

inline float mapRangeClamped(float value, float inA, float inB, float outA, float
outB) {
    return lerp(outA, outB, inverseLerp(inA, inB, value));
}

/* Pacejka Magic Formula (simplified, no Sh/Sv offsets):
 *  $F(x) = D * \sin(C * \operatorname{atan}(B * x - E * (B * x - \operatorname{atan}(B * x))))$ 
 * x is the slip ratio (longitudinal) or slip angle in radians (lateral). */
inline float magicFormula(float slip, float B, float C, float D, float E) {
    float bx = B * slip;
    return D * std::sin(C * std::atan(bx - E * (bx - std::atan(bx))));
}

/* Stiffness factor B so the Magic Formula peaks at |slip| = peakSlip. */
inline float magicStiffnessFromPeak(float C, float peakSlip) {
    if (peakSlip < 1e-6f || C < 1e-6f) return 0.0f;
    return std::tan(PI / (2.0f * C)) / peakSlip;
}

/* Local slope dF/dslip of the Magic Formula at the given slip. Used to treat
 * the tire force implicitly w.r.t. wheel speed (numerical stability). */
inline float magicFormulaSlope(float slip, float B, float C, float D, float E) {
    float bx = B * slip;
    float phi = bx - E * (bx - std::atan(bx));
    float dphi = B * ((1.0f - E) + E / (1.0f + bx * bx));
    float a = std::atan(phi);
    return D * C * std::cos(C * a) * (1.0f / (1.0f + phi * phi)) * dphi;
}

/* Owns its keyframes; evaluates with clamped linear interpolation. */
class Curve {
public:
    Curve() = default;

    void assign(const CP_Curve& c) {
        m_times.clear();
        m_values.clear();
        if (c.count > 0 && c.times && c.values) {

```



```

        m_times.assign(c.times, c.times + c.count);
        m_values.assign(c.values, c.values + c.count);
    }
    m_max = 0.0f;
    bool first = true;
    for (float v : m_values) {
        if (first || v > m_max) { m_max = v; first = false; }
    }
}

float evaluate(float t) const {
    if (m_values.empty()) return 0.0f;
    if (t <= m_times.front()) return m_values.front();
    if (t >= m_times.back()) return m_values.back();
    for (size_t i = 0; i + 1 < m_times.size(); ++i) {
        if (t >= m_times[i] && t <= m_times[i + 1]) {
            float span = m_times[i + 1] - m_times[i];
            float u = span > 1e-12f ? (t - m_times[i]) / span : 0.0f;
            return m_values[i] + (m_values[i + 1] - m_values[i]) * u;
        }
    }
    return m_values.back();
}

float maxValue() const { return m_max; }

private:
    std::vector<float> m_times;
    std::vector<float> m_values;
    float m_max = 0.0f;
};

} // namespace carsim

#endif // CARSIM_MATH_UTIL_H

```

Листинг В.2 — car_sim.h

```

/*
 * car_sim.h - top-level orchestrator that wires the ported models together
 * in the same order the original Unity controllers ran them.
 */
#ifndef CARSIM_CAR_SIM_H
#define CARSIM_CAR_SIM_H

#include "models.h"
#include "car_physics.h"

namespace carsim {

class CarSim {
public:
    bool init(const CP_CarConfig& config);

    void updateDrivetrain(const CP_DrivetrainInput& in, CP_DrivetrainOutput& out);
    void updateWheels(const CP_WheelInput& in, CP_WheelOutput& out);

private:
    EngineModel m_engine;
    GearboxModel m_gearbox;
    ClutchModel m_clutch;
    DifferentialModel m_differential;
    BrakesModel m_brakes;
    SteeringModel m_steering;
};
}

```

```

    SuspensionWheel    m_suspension[CARSIM_WHEEL_COUNT];
    AccelerationWheel  m_acceleration[CARSIM_WHEEL_COUNT];
    SlipWheel          m_slip[CARSIM_WHEEL_COUNT];
    TireWheel          m_tire[CARSIM_WHEEL_COUNT];
    VisualWheel         m_visual[CARSIM_WHEEL_COUNT];

    /* steer angles produced by the drivetrain phase, consumed by visuals */
    float m_steerAngles[CARSIM_WHEEL_COUNT] = { 0 };
};

} // namespace carsim

#endif // CARSIM_CAR_SIM_H

```

Листинг В.3 — car_sim.cpp

```

/*
 * car_sim.cpp - orchestration. The two update phases mirror the order the
 * original Unity controllers executed inside one FixedUpdate:
 *
 *   drivetrain: steering -> gearbox/clutch -> engine -> differential -> brakes
 *   wheels:     visuals -> suspension -> acceleration -> slip -> tire
 *
 * Wheel acceleration deliberately consumes the tire fx from the PREVIOUS tick
 * (one-frame lag), exactly as in the C# version.
 */
#include "car_sim.h"

namespace carsim {

bool CarSim::init(const CP_CarConfig& config) {
    if (config.gearbox.gearCount <= 0 || !config.gearbox.ratios) return false;

    m_engine.init(config.engine);
    m_gearbox.init(config.gearbox);
    m_clutch.init(config.clutch, m_engine.maxEngineTorque());
    m_differential.init(config.differential, config.wheels[0]);
    m_brakes.init(config.brakes);
    m_steering.init(config.steering, config.wheelBase, config.rearTrack);

    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
        m_suspension[i].init(config.wheels[i]);
        m_acceleration[i].init(config.wheels[i]);
        m_slip[i].init(config.wheels[i]);
        m_tire[i].init(config.wheels[i]);
        m_visual[i].init();
        m_steerAngles[i] = 0.0f;
    }
    return true;
}

void CarSim::updateDrivetrain(const CP_DrivetrainInput& in, CP_DrivetrainOutput& out)
{
    float angularVelocities[CARSIM_WHEEL_COUNT];
    float lateralAccelerations[CARSIM_WHEEL_COUNT];
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
        angularVelocities[i] = m_acceleration[i].angularVelocity();
        lateralAccelerations[i] = m_slip[i].lateralAcceleration();
    }

    // gear shift requests (edge-triggered by the host) + timer
    if (in.gearUp)    m_gearbox.requestShiftUp();
    if (in.gearDown) m_gearbox.requestShiftDown();
}

```

```

m_gearbox.tick(in.dt);

// 1. steering
m_steering.update(in.steer, lateralAccelerations, in.dt);
const float* steer = m_steering.steerAngles();
for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) m_steerAngles[i] = steer[i];

// 2. gearbox + clutch
float diffShaftVelocity = m_differential.inputShaftVelocity();
float gearBoxInputShaftVelocity = m_gearbox.inputShaftVelocity(diffShaftVelocity);
m_clutch.update(m_engine.angularVelocity(), m_gearbox.currentGearRatio(),
               gearBoxInputShaftVelocity);

// 3. engine
Vec3 bodyTorque;
bool applyTorque = m_engine.update(in.throttle, m_clutch.torque(),
                                   m_gearbox.currentGear(), in.dt, bodyTorque);

// 4. differential
float gearBoxTorque = m_gearbox.outputTorque(m_clutch.torque());
m_differential.update(gearBoxTorque, angularVelocities, in.dt);

// 5. brakes
m_brakes.update(in.brake, angularVelocities);

// outputs
for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) out.steerAngles[i] =
m_steerAngles[i];
out.neutralBodyTorque = bodyTorque.toC();
out.applyNeutralTorque = applyTorque ? 1 : 0;
out.engineRpm = m_engine.rpm();
out.engineAngularVelocity = m_engine.angularVelocity();
out.currentGear = m_gearbox.currentGear();
out.clutchTorque = m_clutch.torque();
out.clutchLock = m_clutch.lock();
}

void CarSim::updateWheels(const CP_WheelInput& in, CP_WheelOutput& out) {
    const float dt = in.dt;

    // 0. visuals - use this tick's transforms but last tick's spin/length,
    // matching the original controller order (visual ran before suspension).
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
        const CP_WheelState& w = in.wheels[i];
        bool isOpposite = (i % 2 == 0);
        Vec3 visualPos = Vec3(w.position) - Vec3(w.up) *
m_suspension[i].currentLength();
        out.visualPosition[i] = visualPos.toC();
        m_visual[i].update(m_acceleration[i].angularVelocity(), m_steerAngles[i],
                        isOpposite, dt, out.spinEulerX[i], out.steerEulerY[i]);
    }

    // 1. suspension (only grounded wheels)
    Vec3 wheelForce[CARSIM_WHEEL_COUNT];
    for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
        if (in.wheels[i].hit) {
            wheelForce[i] = m_suspension[i].update(in.wheels[i], dt);
        } else {
            wheelForce[i] = Vec3{ 0.f, 0.f, 0.f };
        }
    }

    // 2. wheel acceleration (uses previous-tick tire fx + this-tick drivetrain)
    const float* driveTorque = m_differential.outputTorque();
    const float* brakeTorque = m_brakes.brakeTorque();

```

```

for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
    if (!in.wheels[i].hit) continue;
    m_acceleration[i].update(m_tire[i].fx(), driveTorque[i], brakeTorque[i],
                             m_suspension[i].linearVelocity().z,
                             m_suspension[i].suspensionForce(), dt);
}

// 3. slip forces
for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
    if (!in.wheels[i].hit) continue;
    m_slip[i].update(m_suspension[i].linearVelocity(),
                     m_suspension[i].suspensionForce(),
                     m_acceleration[i].angularVelocity(), dt);
}

// 4. tire forces (accumulate onto the suspension force)
for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
    if (!in.wheels[i].hit) continue;
    Vec3 tireForce = m_tire[i].update(in.wheels[i], m_slip[i].slipRatio(),
                                       m_slip[i].slipAngleRad(),
                                       m_suspension[i].suspensionForce());
    wheelForce[i] = wheelForce[i] + tireForce;
}

// outputs
for (int i = 0; i < CARSIM_WHEEL_COUNT; ++i) {
    out.applyForce[i] = wheelForce[i].toC();
    out.applyPoint[i] = in.wheels[i].hitPoint;
    out.angularVelocity[i] = m_acceleration[i].angularVelocity();
    out.suspensionForce[i] = m_suspension[i].suspensionForce();
    out.currentLength[i] = m_suspension[i].currentLength();
    out.linearVelocity[i] = m_suspension[i].linearVelocity().toC();
    out.slipAngle[i] = m_slip[i].slipAngle();
    out.lateralAcceleration[i] = m_slip[i].lateralAcceleration();
    out.slipForceLong[i] = m_slip[i].slipRatio();
    out.slipForceLat[i] = m_slip[i].slipAngle();
    out.normalizedTireMagnitude[i] = m_tire[i].normalizedMagnitude();
    out.fx[i] = m_tire[i].fx();
}
}

} // namespace carsim

```

Листинг В.4 — api.cpp

```

/*
 * api.cpp - the exported C ABI. Thin: validates handles and forwards to the
 * CarSim orchestrator.
 */
/* CARPHYSICS_BUILD_DLL is defined by the build (vcxproj / CMake) so the API
 * is exported with __declspec(dllexport). */
#include <new>
#include "car_physics.h"
#include "car_sim.h"

using carsim::CarSim;

extern "C" {

CARPHYSICS_API CP_Handle CARPHYSICS_CALL carsim_create(const CP_CarConfig* config) {
    if (!config) return nullptr;
    CarSim* sim = new (std::nothrow) CarSim();
    if (!sim) return nullptr;
    if (!sim->init(*config)) {
        delete sim;
        return nullptr;
    }
}

```

```

    }
    return static_cast<CP_Handle>(sim);
}

CARPHYSICS_API void CARPHYSICS_CALL carsim_destroy(CP_Handle handle) {
    delete static_cast<CarSim*>(handle);
}

CARPHYSICS_API void CARPHYSICS_CALL carsim_update_drivetrain(
    CP_Handle handle, const CP_DrivetrainInput* in, CP_DrivetrainOutput* out) {
    if (!handle || !in || !out) return;
    static_cast<CarSim*>(handle)->updateDrivetrain(*in, *out);
}

CARPHYSICS_API void CARPHYSICS_CALL carsim_update_wheels(
    CP_Handle handle, const CP_WheelInput* in, CP_WheelOutput* out) {
    if (!handle || !in || !out) return;
    static_cast<CarSim*>(handle)->updateWheels(*in, *out);
}

CARPHYSICS_API const char* CARPHYSICS_CALL carsim_version(void) {
    return "1.0.0";
}

} // extern "C"

```

ПРИЛОЖЕНИЕ Г

(обязательное)

Интеграция модуля в игровой движок Unity

Листинг Г.1 — CarPhysicsNative.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.InteropServices;
using Car.Data;
using UnityEngine;

namespace Car
{
    /// <summary>
    /// Thin P/Invoke binding over CarPhysics.dll (the native, engine-agnostic
    /// car-simulation module). Mirrors include/car_physics.h. All the physics
    /// math lives in the DLL; this file only marshals data in and out.
    /// </summary>
    public static class CarPhysicsNative
    {
        private const string DLL = "CarPhysics";
        public const int WheelCount = 4;
        public const float RPM_TO_RAD = Mathf.PI * 2f / 60f;
        public const float RAD_TO_RPM = 1f / RPM_TO_RAD;

        // ----- structs (layout identical to car_physics.h) -----

        [StructLayout(LayoutKind.Sequential)]
        public struct Vec3
        {
            public float x, y, z;
            public Vector3 ToUnity() => new Vector3(x, y, z);
            public static Vec3 From(Vector3 v) => new Vec3 { x = v.x, y = v.y, z = v.z };
        };

        [StructLayout(LayoutKind.Sequential)]
        private struct Curve { public IntPtr times; public IntPtr values; public int count; }

        [StructLayout(LayoutKind.Sequential)]
        private struct EngineInfo
        {
            public Curve torqueCurve;
            public Vec3 engineOrientation;
            public float idleRpm, maxRpm, mul, frictionCoeff, startFriction, inertia;
        }

        [StructLayout(LayoutKind.Sequential)]
        private struct GearboxInfo { public IntPtr ratios; public int gearCount; public float shiftTime; }

        [StructLayout(LayoutKind.Sequential)]
        private struct ClutchInfo { public float stiffness, capacity, damping; }

        [StructLayout(LayoutKind.Sequential)]
        private struct DifferentialInfo { public int isLocked; public float ratio; }
```

```

[StructLayout(LayoutKind.Sequential)]
private struct BrakesInfo { public Curve brakeTorqueCurve; public float
maxTorque, biasFront, biasRear; }

[StructLayout(LayoutKind.Sequential)]
private struct SteeringInfo { public float turnRadius, steerForce,
maxCorrectionAngle, correctionSpeed; }

[StructLayout(LayoutKind.Sequential)]
private struct WheelInfo
{
    public float restLength, suspensionStiffness, damperStiffness,
slipAnglePeak,
        camber, caster, longitudinalCoeff, lateralCoeff, wheelRadius,
        wheelMass, longFrictionCoeff, relaxationLength;
    // Pacejka Magic Formula shape parameters (must match CP_WheelInfo order).
    public float longSlipPeak, pacejkaShapeLong, pacejkaCurveLong,
        pacejkaShapeLat, pacejkaCurveLat;
}

[StructLayout(LayoutKind.Sequential)]
private struct CarConfig
{
    public EngineInfo engine;
    public GearboxInfo gearbox;
    public ClutchInfo clutch;
    public DifferentialInfo differential;
    public BrakesInfo brakes;
    public SteeringInfo steering;
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)]
    public WheelInfo[] wheels;
    public float wheelBase, rearTrack;
}

[StructLayout(LayoutKind.Sequential)]
public struct DrivetrainInput
{
    public float dt, throttle, brake, steer;
    public int gearUp, gearDown;
}

[StructLayout(LayoutKind.Sequential)]
public struct DrivetrainOutput
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)]
    public float[] steerAngles;
    public Vec3 neutralBodyTorque;
    public int applyNeutralTorque;
    public float engineRpm, engineAngularVelocity;
    public int currentGear;
    public float clutchTorque, clutchLock;
}

[StructLayout(LayoutKind.Sequential)]
public struct WheelState
{
    public Vec3 position, right, up, forward;
    public int hit;
    public Vec3 hitPoint, hitNormal, pointVelocity;
}

[StructLayout(LayoutKind.Sequential)]
public struct WheelInput
{
    public float dt;

```

```

        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)]
        public WheelState[] wheels;
    }

    [StructLayout(LayoutKind.Sequential)]
    public struct WheelOutput
    {
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)] public Vec3[] applyForce;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)] public Vec3[] applyPoint;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)] public Vec3[] visualPosition;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] spinEulerX;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] steerEulerY;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] angularVelocity;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] suspensionForce;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] currentLength;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount, ArraySubType
= UnmanagedType.Struct)] public Vec3[] linearVelocity;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] slipAngle;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] lateralAcceleration;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] slipForceLong;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] slipForceLat;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] normalizedTireMagnitude;
        [MarshalAs(UnmanagedType.ByValArray, SizeConst = WheelCount)] public
float[] fx;
    }

    // ----- exported entry points -----

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    private static extern IntPtr carsim_create(ref CarConfig config);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    public static extern void carsim_destroy(IntPtr handle);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    public static extern void carsim_update_drivetrain(IntPtr handle, ref
DrivetrainInput input, out DrivetrainOutput output);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    public static extern void carsim_update_wheels(IntPtr handle, ref WheelInput
input, out WheelOutput output);

    [DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
    [return: MarshalAs(UnmanagedType.LPStr)]
    public static extern string carsim_version();

    // ----- convenience: build a sim from a CarDesc -----

    private const int CurveSamples = 64;

```



```

    /// <summary>Creates a native sim from the scriptable CarDesc. Returns the
opaque handle.</summary>
    public static IntPtr Create(CarDesc desc, float wheelBase, float rearTrack)
    {
        var pins = new List<GCHandle>();
        try
        {
            var cfg = new CarConfig
            {
                engine = new EngineInfo
                {
                    torqueCurve = MakeCurve(desc.engineInfo.torqueCurve, pins),
                    engineOrientation =
Vec3.From(desc.engineInfo.engineOrientation),
                    idleRpm = desc.engineInfo.engineIdleRpm,
                    maxRpm = desc.engineInfo.engineMaxRpm,
                    mul = desc.engineInfo.engineMul,
                    frictionCoeff = desc.engineInfo.engineFrictionCoefficient,
                    startFriction = desc.engineInfo.startFriction,
                    inertia = desc.engineInfo.engineInertia,
                },
                gearbox = MakeGearbox(desc.gearboxInfo, pins),
                clutch = new ClutchInfo
                {
                    stiffness = desc.clutchInfo.clutchStiffness,
                    capacity = desc.clutchInfo.clutchCapacity,
                    damping = desc.clutchInfo.clutchDamping,
                },
                differential = new DifferentialInfo
                {
                    isLocked = desc.differentialInfo.isDiffLocked ? 1 : 0,
                    ratio = desc.differentialInfo.differentialRatio,
                },
                brakes = new BrakesInfo
                {
                    brakeTorqueCurve = MakeCurve(desc.brakesInfo.brakeTorqueCurve,
pins),

                    maxTorque = desc.brakesInfo.maxTorque,
                    biasFront = desc.brakesInfo.brakeBias[0],
                    biasRear = desc.brakesInfo.brakeBias[1],
                },
                steering = new SteeringInfo
                {
                    turnRadius = desc.steeringInfo.turnRadius,
                    steerForce = desc.steeringInfo.steerForce,
                    maxCorrectionAngle = desc.steeringInfo.maxCorrectionAngle,
                    correctionSpeed = desc.steeringInfo.correctionSpeed,
                },
                wheels = new WheelInfo[WheelCount],
                wheelBase = wheelBase,
                rearTrack = rearTrack,
            };

            for (int i = 0; i < WheelCount; i++)
            {
                var w = desc.wheelInfos[i];
                cfg.wheels[i] = new WheelInfo
                {
                    restLength = w.restLength,
                    suspensionStiffness = w.suspensionStiffness,
                    damperStiffness = w.damperStiffness,
                    slipAnglePeak = w.slipAnglePeak,
                    camber = w.camber,
                    caster = w.caster,
                    longitudinalCoeff = w.longitudinalCoeff,
                    lateralCoeff = w.lateralCoeff,
                    wheelRadius = w.wheelRadius,
                    wheelMass = w.wheelInertia / (w.wheelRadius *
w.wheelRadius), // recover mass
                    longFrictionCoeff = w.longFrictionCoeff,

```

```

        relaxationLength = w.relaxationLength,
        longSlipPeak = w.longSlipPeak,
        pacejkaShapeLong = w.pacejkaShapeLong,
        pacejkaCurveLong = w.pacejkaCurveLong,
        pacejkaShapeLat = w.pacejkaShapeLat,
        pacejkaCurveLat = w.pacejkaCurveLat,
    };
}

return carsim_create(ref cfg);
}
finally
{
    foreach (var h in pins) h.Free();
}
}

private static Curve MakeCurve(AnimationCurve curve, List<GCHandle> pins)
{
    // Sample Unity's (Hermite) curve into a dense piecewise-linear table
    // so the DLL's linear evaluation closely matches the editor curve.
    float[] times, values;
    if (curve == null || curve.length == 0)
    {
        times = new float[] { 0f };
        values = new float[] { 0f };
    }
    else
    {
        float t0 = curve.keys[0].time;
        float t1 = curve.keys[curve.length - 1].time;
        int n = curve.length == 1 ? 1 : CurveSamples;
        times = new float[n];
        values = new float[n];
        for (int i = 0; i < n; i++)
        {
            float t = n == 1 ? t0 : Mathf.Lerp(t0, t1, i / (float)(n - 1));
            times[i] = t;
            values[i] = curve.Evaluate(t);
        }
    }
    return new Curve { times = Pin(times, pins), values = Pin(values, pins),
count = times.Length };
}

private static GearboxInfo MakeGearbox(CarDesc.GearboxInfo info,
List<GCHandle> pins)
{
    var ratios = info.gearBoxRatios.ToArray();
    return new GearboxInfo { ratios = Pin(ratios, pins), gearCount =
ratios.Length, shiftTime = info.shiftTime };
}

private static IntPtr Pin(float[] data, List<GCHandle> pins)
{
    var h = GCHandle.Alloc(data, GCHandleType.Pinned);
    pins.Add(h);
    return h.AddrOfPinnedObject();
}
}
}

```